Client internals and version profiles, part 3 of 4

Client internals and version profiles, part 3 of 4

This part contains 24 notes, from `client-internals/graalconfig-reference.md` through `client-internals/rendering-observations.md`.

`client-internals/graalconfig-reference.md`

graalconfig.txt and registry reference (client 2.05)

Date: 2026-10-21

Traced from the 2.05 rev 2 options reader and writer, the More Options and Player List Options dialogs, and their form resources. Checked against a real `graalconfig.txt` written by the 2.22 client, which uses the same keys in the same order. Tags: **[2.05]** read from the client · **[2.22]** seen in a 2.22 file · **[inf]** inferred.

Where settings live

Store	Location	Holds
`graalconfig.txt`	Next to the exe	Every option on the More Options and Player List Options dialogs, the head, language, font, joystick and key bindings
Registry	`HKEY_CURRENT_USER\Software\Graal\Client`	`nickname`, `accountname`, `password`, and one `<account>:pass` value per account

The file is a plain `key=value` list, read with Delphi `TStringList.Values`. Key lookup is case-insensitive, and order doesn't matter on read. **[2.05]**

When it is read and written

- **Read** once, when the options form is created at startup. **[2.05]**
- **Written** in full every time the More Options or Player List Options dialog closes. The writer always emits every key in the fixed order below, so hand edits that add other keys are lost on the next save. **[2.05]**
- **If the file is missing**, the client keeps its built-in defaults and sets `language` to the English name of the Windows user locale (for example `English` or `German`). **[2.05]**

Value parsing

Kind	Rule	Pitfall
Boolean	`true` when the value, lowercased, is exactly `true`. Anything else is `false`. Written as `true` / `false`.	A **missing** boolean key reads as `false`, not as the default. A partial file therefore switches off the options that default to on: `musicon`, `soundfxon`, `saveimages`, `nohtmlimages`, `joystickon`.
Integer	`StrToInt`. A conversion error is caught and the default is kept.	An empty or missing integer key keeps the default.
String	Used only when non-empty.	

Keys, in file order

UI labels are the dialog captions. MO = More Options, PL = Player List Options.

Key	Type	Default	UI	Effect
`waitvertical`	bool	false	MO "Wait for vertical blank (slower but smoother)"	Vsync on flip
`musicon`	bool	true	MO "Music support"	Turning it off stops the music player **[inf]**
`usewinamp`	bool	false	MO "Use WinAmp for music"	Music goes through Winamp
`soundfxon`	bool	true	MO "Sound FX's enabled"	
`reversestereo`	bool	false	MO "Reverse stereo channels"	
`saveimages`	bool	true	MO "Save downloaded images (to)"	Turning it on writes the images already in memory to disk **[inf]**
`dontsavepms`	bool	false	PL "Don't save PMs/ Profiles"	
`compabilitydraw`	bool	false	MO "Draw in compatibility mode (very slow)"	Spelling as in the client
`nodecompressionerror`	bool	false	MO "Don't show decompression errors"	
`head1`, then `head`	string	`Fung Si Yun`	none	Read in that order, so `head` wins. Only `head=` is written. The default value is a literal in the client. **[2.05]** The 2.22 file still holds it. **[2.22]**
`joystickon`	bool	true	MO "Joystick support"	
`nodirectinput`	bool	false	MO "Don't use DirectInput"	
`joyselected` (also reads `joyselected1`)	string	`(none)`	Controls frame	`(none)`, `Joystick1` or `Joystick2`
`keyset_<action>_joystickbutton`	string	per action	Controls frame	One of `Joy_Up`, `Joy_Down`, `Joy_Left`, `Joy_Right`, `Button1`… `Button12`
`keyset_<action>_key0` … `_key2`	string	per action	Controls frame	Three key slots per action, by key name (`Up`, `Num 8`, `Space`, `Tab`, …). Empty means unbound.

Key	Type	Default	UI	Effect
`language`	string	`English`	Connect frame language box	
`font`	string	`Tempus Sans ITC`	MO "Font:"	Chat and name font. The 2.22 folder ships this font file. Defaults per build in `chat-text.md`
`fontsize`	int	24	MO "Size:"	
`fixedport`	bool	false	MO "Use fixed port for data packages:"	
`fixedudpport`	int	14899	MO port box	Used when `fixedport` is true
`noudp`	bool	false	MO "Don't use UDP for data packages"	Changing the UDP options restarts the UDP socket **[inf]**
`nomassmessages`	bool	false	PL "Ignore mass messages"	
`nohtmlinpms`	bool	false	PL "Don't use HTML for PMs"	
`limitnicknames`	bool	false	MO "Limit the lenght of nicknames to"	
`nicknamelimit`	int	50	MO length box	
`dontconnectlevels`	bool	false	MO "Don't connect levels"	Turning it on drops the cached neighbour levels **[inf]**
`notoalls`	bool	false	MO "Ignore toalls"	Turning it on also clears the toall history **[inf]**
`nohtmlimages`	bool	true	PL "No images in PMs"	
`sortplayersbyname`	bool	false	PL "Playerlist: sort by nickname"	
`dontloadlistheads`	bool	false	MO "Don't load heads for the playerlist / map"	
`dontsavepasswords`	bool	false	MO "Don't save the passwords"	See below
`dontusedirectx7`	bool	false	MO "Don't use DirectX7"	
`fullscreenmode`	string	`1024x768`	MO "Fullscreen mode:"	One of `640x480`, `800x600`, `1024x768`, `1280x1024`, `1600x1200`. Any other value keeps the default resolution.

The joystick and key block is written right after `head=`. The 11 actions, in file order, are `up`, `left`, `down`, `right`, `item`, `attack`, `action`, `map`, `switchtochat`, `itemlist`, `pause`. **[2.05]**

Key binding rules on load: **[2.05]** - A key name that is already bound earlier in the file is treated as `-` (unbound), so one key can't drive two slots. - Binding a key that another slot already holds moves it, and the other slot gets the old key.

Default joystick buttons, in action order: `Joy_Up`, `Joy_Left`, `Joy_Down`, `Joy_Right`, `Button1`…`Button5`, `Button9`, `Button10`. **[2.05]** The 2.22 file matches. **[2.22]**

Keys in older notes that 2.05 does not read

`serveraddr`, `loadcolorsfromserver` and `gamemode` (listed in `config-and-input.md`) are not read or written by 2.05. If they exist, they belong to later clients. **[2.05]**

Registry values

Value under `HKCU\Software\Graal\Client`	Read	Written
`nickname`	At startup	When the dialogs close
`accountname`	At startup. It is a comma list, and the first entry is the current account.	When the dialogs close
`password`	At startup, decrypted	When the dialogs close, unless `dontsavepasswords` is true

Value under `HKCU\Software\Graal\Client`	Read	Written
`<account>:pass`	At startup for the current account, and again when the account box changes	Same as `password`

When the More Options dialog closes with "Don't save the passwords" ticked, the client deletes `password` and every value whose name ends in `:pass` from that key. **[2.05]**

Password values are encrypted with a fixed key. See `saved-passwords.md`.

`client-internals/id24-205.md`

Id 24 in 2.05: the same weapon-image list

In 2.05, client id 24 already carries the weapon-script image list that the 2.17.1, 2.22 and 2.31 clients send (`decomp-findings-2026-10-25-g/01-id24-image-list.md`). It is not an emote, and it has no other meaning. Read from the 2.05 program.

What 2.05 sends

- **The images:** the images that the selected weapon's script shows, at indexes 0 to 199. A single image set aside by the client replaces that list when present.
- **Changes only:** the list is compared with the one sent last.
  - An image that is new or has changed is sent as a record.
  - An image that is gone is sent as its index byte followed by an empty name.
- **Record layout:** a record starts with 0x2a plus the image's index. Tag bytes follow (0x20 plus the tag), as in the later builds:
  - tag 0 is the name;
  - tags 1 and 2 are x and y;
  - tag 3 is the layer, sent only when it is not 1;
  - tag 4 is the part, sent only when its width and height are above 0;
  - tag 5 is the colours, sent only when any is not 1.0;
  - tag 6 is the zoom, sent only when it is not 1.0.
- **Full list:** the full list starts with the marker 0x29 (GCHAR 9), as later. It goes out:
  - when nothing has been sent yet and there are images;
  - and when a countdown of 60 runs out. That is every 61st call, against 101 in the later builds.
- **Copies:** the text always goes out as UDP id 15. The TCP copy (id 24) goes out when UDP is off for the client, or when some player in the level is not reached by UDP. This is the same test as 2.17.1 and 2.22.
- **Empty text:** nothing is sent.

Correction

Earlier notes said 2.05 and earlier use "a four-field form (index, x, y, name)" for this id. That is wrong for 2.05: it uses the tagged form above, with tags 0 to 6. Tags 7 and 8 of the later builds are not sent by 2.05.

Confirmed, inferred, not tested

- Confirmed from the program: the source of the images, the diff, the record start, the tags and when each is sent, the marker, the countdown, and the UDP/TCP split.
- Inferred: that the countdown runs on the same periodic handler as in the later builds, which would give about 3 s.
- Not tested on a wire. Builds before 2.05 were not read.

`client-internals/item-drops.md`

Item drops from bushes, pots and stones, per client version

Date: 2026-10-30

What a cut or lifted bush, a lifted pot and a stone leave, who rolls it, and what the client sends about items on the floor. Covers every build from the 4th alpha to 2.31, read in the binaries and checked in the lab with real 1.39, 2.05 rev 2 and 2.22 clients. This corrects parts of `board-respawn.md` (see the end).

Tags: **[bin]** read from the binary · **[live]** seen in the lab with the real client under Wine · **[cap]** read from a lab capture · **[src]** GServer-v2 source · **[doc]** `newfeatures*.txt` · **[inf]** inferred.

The captures, their decoded message lists and screenshots are in the project files under `graal-decompile/live-trace/item-drops/`.

Summary

Builds	Who drops	What the client sends
4th alpha to 1.37	The client rolls **[bin]**	Roll: UDP 19 to the other players only. Pickup: applied locally, UDP 20 and TCP `PLI_ITEMDEL` (13). Timeout: nothing **[bin]**
1.38 to 1.41	The client rolls **[bin]**	Roll: UDP 19 and TCP `PLI_ITEMADD` (12). Pickup online: `PLI_ITEMTAKE` (32), and the server gives the item with props. Timeout online: `PLI_ITEMDEL` (13) **[bin][cap]** for 1.39; 1.41 read for the roll only
1.42 (NewMain) to 2.31	The server (`PLO_ITEMADD`, 22). Offline nothing drops **[bin][doc]**	Pickup: `PLI_ITEMTAKE` (32). Timeout: `PLI_ITEMDEL` (13) **[bin][cap]**

- **The change is at 1.42, not 2.00.** NewMain 1.42's `newfeatures142.txt` says "bush rupees and pot hearts are server-side", and the roll is gone from NewMain 24 and 26 **[doc][bin]**. 1.41 (graal_gr1411) still has it **[bin]**.
- **No client ever rolls arrows**, and no client drops anything for a stone **[bin]**. Arrows from a bush come only from a server that has them in its table (GServer-v2: `rand() % 6` includes darts **[src]**).
- **Stones.** The only client path that touches a stone is the bomb blast, which destroys a sign, pot or stone and gives a heart only for the pot **[bin]**. "Something with rocks" is most likely bombs breaking them.

The 1.x roll [bin]

One `Random(100)` per drop, with Delphi's `Random` (`seed = seed * 0x08088405 + 1`, result `(seed * range) >> 32`):

Roll	Item
0 to 24	green rupee (25 %)
25, 26	blue rupee (2 %)
27 to 31	heart (5 %)
32 to 36	bombs (5 %)
37 to 99	nothing (63 %)

- The same table is in every build from the 4th alpha to 1.41. From 1.38 the code picks the item by its number in the item table (0, 1, 5, 3) instead of by name.
- **Sword cut:** every entry of the cut table rolls, tall grass too (1.3 and 1.41 read). `board-respawn.md` said bush only.
- **Lift:** lift entry 0 (bush) rolls; entry 2 (pot) always leaves a heart; sign and stones leave nothing.
- **Bomb blast** (`TAdventureExplosion`, four test points): cut entry 0 rolls; lift entries 1 to 3 (sign, pot, stone) are destroyed, and the pot leaves a heart.
- The item lies at the object's top-left tile (1.39 capture: the bush at 31, 34 sent `^d`).
- A new item gets life 164 and the "announce" flag.
- `bushitems` in GServer-v2 beta4's `serveroptions.txt` says the same thing from the server side: "If making a 1.41 server, set bushitems, vasesdrop, and baddyitems to false as the 1.41 client generates items."

Items on the floor (2.05 rev 2) [bin]

The item object (`TAdventureExtra`) and the level's item tick:

- **Life.** Every 50 ms tick the life goes down by 1, from 164. Below 0 the item goes; online the client sends `PLI_ITEMDEL`. Lab: 8.20 s from the drop to the 13 **[cap]**.
- **Pickup** when all hold: the item is not permanent (life below 100 at creation, such as a chest's display item with life 24), the life is below 160 (so not in the first four ticks), and `|px + 1.5 - (ix + 1)| < 2` and `|py + 2 - (iy + 1)| < 2`.
- At life 140 the `playerlaysitem` event fires.
- **Blink:** not drawn while the life is below 37 and odd.
- **Bounce:** raised by `|sin(life / 3)| · (life / 164)^16 · 3` tiles while the life is at least `164 - 7π`.
- **Draw:** the shadow, sprite 42, at `(x + 0.5, y + 1 + h / 2 - 0.5)` for an item `h` tiles high; the item centred on `(x + 1, y + 1)`.
- **Sprites by item number:** 136, 137, 138, 67, 143, 139, 145, 351, 65, 350, 314, 315, 321, 323, 322, 324, 325, 328, 329, 330, 331, 332, 333, 334, 349.
- **Floor items** (the announce flag; `PLO_ITEMADD` makes an item only for these): 0 to 5, 7, 8, 19 and 24 (rupees, bombs, darts, heart, bow, bomb, gold rupee, spin attack).
- **What a pickup gives offline:** green 1, blue 5, red 30, gold 100 rupees (up to 9,999,999); bombs and darts 5 each (up to 99); a heart one whole heart. Sound `extra.wav` (1.3 and 1.39: `sounds\extra.wav`).

1.3 differs **[bin]**: the pickup test is an overlap of pixel boxes, and a timeout is silent. Whether 1.3's life count stops behind a flag (offset 0x74) was not settled.

Lab runs [live][cap]

Client and server	What happened
2.05 rev 2, old lab pack (2.05 in the 2.1 slot, `tiledroprate` 100)	Cut: C>S 1, then S>C 7 and S>C 22 `^d"` (red rupee at 31, 34). Left alone: C>S 13 `^d` 8.20 s later. Picked up: C>S 32 `^d!`, then S>C 9 RUPEESCOUNT 5, and the HUD showed 5. A lifted pot: S>C 22 heart
2.22, beta4	The same: 22, then 13 after about 8.7 s; pickup 32, then props 9
2.05 rev 2, beta4	No drops: beta4 counts 2.05 as `CLVER_2_052`, below 2.1, and GServer-v2 gives no drops below 2.1 **[src]**
1.39, beta4 (`generation = classic`)	The client rolled: C>S 1, then C>S 12 `^d` in the same frame. Picked up: C>S 32 `^d`. The server did not echo 22 to the sender. Four of the next five cuts dropped nothing

Seihai against the same servers (2026-10-30) [cap]

- **2.05:** cut, S>C 22 bombs, drawn with its shadow and bounce; C>S 13 8.39 s later. A heart: picked up after 1.09 s with C>S 32, S>C 9 CURPOWER.
- **1.39:** lift, C>S 1 and C>S 12 `^\x82` (green rupee) in the same frame; C>S 32 on pickup; S>C 9 RUPEESCOUNT, and the HUD count went up.

Corrections to `board-respawn.md`

- Drops became server-side at **1.42**, not 2.00.
- The 1.x cut roll is for every cut entry, not the bush alone.
- From 1.38 the roll also goes to the server (`PLI_ITEMADD`), not only to peers.
- The drop tables of 1.39 to 1.41 were read: the same as 1.3.

Open

- 1.41's pickup and timeout path is assumed to be 1.39's.
- 1.x pickup box in tiles (only 1.3's pixel overlap was read).
- Whether items are drawn under or over players (Seihai draws them under, inferred).

`client-internals/lab-captures-2026-10-25/01-damage-chest-respawn-weapons.md`

Lab captures 2026-10-25: chests, damage, CURPOWER, respawn, default weapon ids (2.22, derived facts only)

Experiments: A chests, B damage powers, C CURPOWER and packet 40, D end-of-respawn save, E default weapon ids. Verdicts are marked settled or OPEN in each section. Method: see the end of this file.

Part 0: facts decoded from earlier-attempt leftovers (labm/res/*.out, real 2.22 client c1 as victim; server->client packets injected through proxy2.py; c2 = a second real 2.22 client, other player id 3)

Harness caveat: positions are read from the client's own X/Y props; hearts are also read from screenshots.

C. CURPOWER semantics (settled for 2.22)

- CURPOWER (prop 0x22) is in HALF hearts; MAXPOWER (prop 0x21) is in hearts. Injecting MAX=10, CUR=28 then taking a half-heart hit made the client send CUR=19: the client clamps CUR to MAX*2 (20) before subtracting. MAX=3, CUR=10 then a half-heart hit sent CUR=5 (clamp to 6, minus 1); the screen showed 2.5 hearts.
- The client does not send anything when the server sets CUR/MAX (no echo).
- After a hit the client sends ONE props message: CURPOWER (new value) + GANI `hurt` + one more prop (0x31). Our own CURPOWER write on injected CUR=0: client sends `dead` GANI and prop 0x32, no CURPOWER.
- After the death key (arrow key) the client sends CURPOWER = hearts by AP bracket (MAX=10): AP 10 and 19 -> 6 (3 hearts); AP 20 and 39 -> 10 (5 hearts); AP 40, 60, 100 -> 20 (10 hearts = full). (lab result file)

B. Packet 40 (hurt player), real client as victim (settled for the cases below)

- Payload as accepted: attacker id (2 bytes), dx byte, dy byte, power byte. Hit applied only when the attacker id is a player the client knows (ids 99, 0, 2 here: ignored, no damage, no knockback).
- Power byte (byte-0x20) is in HALF hearts: 0 -> knockback only, no hearts; 1 -> half heart (CUR 6->5); 2, 3, 4, 15, 31 -> one heart (CUR 6->4). So the cap is 2 half hearts, matching the decompile ("capped at one heart").
- Knockback: direction from (dx-0x60, dy-0x60); always 5 tiles (50 ms steps); diagonal (0x70,0x70) gives (3.5, 3.5) so the vector is normalised. 0x61/0x68 gave the same as 0x70 (sign/direction only), 0x5f/0x50 went the other way.
- A trailing 4-byte field (sent as 20 20 20 20) made the hit do NOTHING (no damage, no knockback); 2 trailing bytes were still accepted, 3 were not. So the NPC-id field (when present as the 4th extra byte) is looked up: an unknown NPC id 0 drops the hit. (Open: what a real NPC id does.) The earlier distance test (exp40d) used the 4-byte form, so its "no hit at distance^2 81..132" is NOT evidence for a distance gate; re-test is in Part 1.
- Server-sent bomb (PLO 11) and arrow (PLO 19) injected near the victim: half-heart hit (CUR 6->5) with a 5-tile knockback directly away from the bomb/arrow start. Damage showed 4 (a full heart) in a few runs (bomb field 0x23, 0x2b; arrow last byte 0x24) without a clear rule: OPEN.
- Real arrow shot by c2 (last byte 0x21 on the wire) never hurt c1 in realarrow2, and an injected arrow with last byte 0x20-0x22 did nothing while 0x23-0x2c hurt: OPEN what that byte means.
- (superseded by the Part 2 bomb notes below) Bomb hit zone from the first sweep was misread.

D. End-of-respawn save (open, partial)

- After death the client sends: CURPOWER 0 + GANI dead; arrow key -> a warp request (client msg id 30) carrying (x,y,level) = (30.0, 30.5, lv_a.nw) = the level's login/entry point, not the death spot (26.0 / 23.5). Second death after the server's warp reply (with the reply delayed 8 s or not delayed): request again (30.0, 30.5). So after the reply arrives the saved point is the entry point either way.
- NOT tested: dying again inside the delay window (before the reply), which is the only case where "save at end of respawn" would show the death spot.

Part 1 (fresh runs, 2026-10-25 ~22:50-23:10 JST): A. Chests (real 2.22 client, labm lab, PLO_LEVELCHEST (id 4) injected)

- Full (closed) form `00 x y item sign` (each +32) is drawn as a closed chest, short form `01 x y` as an OPEN chest (dark inside). Each is a 32x32 px block = 2x2 tiles at x,y (top-left), drawn over the grass board; no board tile needed. Settled for 2.22.
- A closed chest is solid like a wall: walking into it from any side stops the player when the 12 body probes touch its 2x2 box (matches #61). Pushing against it plays `push.gani`.
- OPENING: only when the player faces UP (the chest is above, the player below it) and presses the action key (A). Facing down, left or right with the player stopped against the chest and A pressed: NOTHING is sent (3 sides tested, each with a chest adjacent). Settled for 2.22; 2.05 not checked.
- On opening (facing up) the client at once: asks for `chest.wav` (WANTFILE), sends PLI_OPENCHEST (20) with x,y = the chest's top-left tile (whole tiles, +32), and draws the chest open locally (no server reply needed). About 1.2 s later it sends packet 13 with x*2*,*y*2 = (chest.x, chest.y-1), i.e. one tile above the chest. It does NOT send a rupee/props update; the rupee counter stayed 0 (the lab server did not know the injected chest, so the reward cannot be judged here). Open: what the server sends back for a known chest.
- Compare Seihai (documentation/chest-drawing.md, #65): drawing and short/full forms agree. Difference: Seihai opens a chest from any facing (offline: the point one tile ahead of the feet); the real 2.22 client only from below facing up.

B/C addendum (fresh runs 23:10-23:30 JST): packet 40 distance gate

- Clean packet (attacker id 3, dx dy power, no trailing bytes), victim c1 on the same row, attacker c2 fixed at x=33.0: hit (CUR 6->4, knockback) at squared distance 64, 81, 90.25, 100.0; NO hit (hearts unchanged, no knockback) at 110.25 and 121.0. So the gate is distance^2 <= 100 (10 tiles), settled for 2.22. The earlier "no hit at 81" results used the NPC form and are explained by the NPC rule below.
- NPC field: the trailing field is 3 bytes (GINT3 npc id). With 3 or 4 trailing bytes (`20 20 20` = id 0, `20 20 21` = id 1, `2a 2a 2a`) the real client refuses the hit completely (no damage, no knockback); with 2 trailing bytes it still hits. This is exactly Seihai's `parse_hurtplayer` (npc only when >= 6 bytes follow the id) plus `Reject::NpcSource`.
- Zero knockback vector (dx=dy=0x60): knocked 5 tiles EAST in every run. Matches the decompile ("a zero vector is replaced by one tile east") and #55.
- Everything else in packet 40 (power cap 2 half hearts, power 0 = knockback only, unknown attacker refused, 5-tile normalised knockback) already matches Seihai #55: no Seihai change needed for packet 40.

D. End-of-respawn save (settled for 2.22, same-level respawn)

Run respawn5.py: die at x=26 (hearts 1), hold the server's reply for 9 s, press an arrow key (warp request (30.0, 30.5, lv_a.nw) at +3.1 s), then INSIDE the 9 s window reset hearts, die again at x=28.5 and press a key: the second warp request is again (30.0, 30.5, lv_a.nw), not the death spot (28.5 / 26). After the held replies arrive the player is at (30.0, 30.5). So the respawn routine's extra save does not make the next respawn point the death spot, with or without the delayed reply. Seihai's current behaviour (respawn at the level's entry point; no end-of-respawn save) agrees. (Seihai documentation said "needs capture"; update the doc.)

E. Default weapon ids -> images (real 2.22; only what the captures show)

- Login with the server's PLO_DEFAULTWEAPON (43) frames filtered to id 7 alone: the client asks `wbowi1.png` (then `wbomb1.png`). Login with ids filtered to none, or to 8 alone: `wbomb1.png` is asked and

`wbowi1.png` is NOT. So id 7 -> `wbowi1.png` is settled; `wbomb1.png` is asked regardless of id 8 (it comes with the client's own bomb item), so id 8 -> `wbomb1.png` is NOT shown by the capture.
- Ids sent after login (one per second, ids 0..39): the client asked for a file only for: 12 -> `wbomb2.png`, 20 -> `wbowi2.png`, 21 -> `wbowi3.png`, 22 -> `wbowi4.png`, 23 -> `wbomb3.png`. Ids 7 and 8 asked nothing (already asked at login); all other ids 0..39 asked nothing. So the table above (7, 12, 20, 21, 22, 23) is all that is known; "bow = 7, bomb = 8" from the server source is only half confirmed (7 yes, 8 not isolated).

B. Bombs, arrows, baddies: who owns the damage (Part 2, fresh runs)

- No hit packet is sent. Across every bomb and arrow trial (leftover logs plus new runs, >100 hits) the real client sent no PLI 26 (hurt player), 46 (hit objects) or 41; the only messages after a hit were the props message (CURPOWER + `hurt` GANI) and the knockback walk. The victim's own client judges the damage when a server-sent bomb (PLO 11) explodes near it or a server-sent arrow (PLO 19) reaches it, whoever owns it. A client's own bomb hurts the owner too (c2's own bomb: c2 lost half a heart, knocked 5 tiles away; each trial 1 half heart).
- Damage seen: 1 half heart (CUR 6->5) with a 5-tile knockback away from the bomb/arrow (victim centre minus the blast point), in every deterministic trial; a few runs showed 4 (a full heart) without a rule: OPEN (maybe a second overlapping hit).
- Bomb zone with a 7.5 s wait (explosion done): victim at (30,30.5), bomb x=30: hits at y offsets +1, +3, -3 (bomb y 31.5, 33.5, 27.5); NO hit at 0, +2, -2 (30.5, 32.5, 28.5). With bomb y = victim y+1: hits at x offsets +4, +5, -4. Row offset 0: nothing at x offset 0, +1, -1. This is not a plain radius; it looks like a pattern keyed on the bomb's half-tile y (the first, 4.5 s-wait sweep had hits at every .0/.5 offset except the integers 0, +-2): OPEN, needs the 2.05/2.22 explosion routine, not guessed here.
- Arrows: an injected PLO 19 arrow (owner 3) hurt only when its last byte was 0x23 or more; bytes 0x20 to 0x22 did nothing; the real arrow c2 fires carries 0x21 and never hurt c1: OPEN (probably a flight/lifetime field).
- Baddies: not tested (the lab server has no baddies). OPEN.
- What Seihai should do: nothing changes for bombs/arrows yet; documentation/damage-respawn.md can say "judged by the victim, half heart, 5-tile knockback away from the blast; blast shape open".

Seihai changes that came out of this

- Chests open only facing up: squirelworks/seihai#73.
- Login asks the images of default weapon ids 7, 12, 20-23: squirelworks/seihai#74.
- Packet 40 (range, cap, NPC refusal, zero vector), CURPOWER clamping and the respawn hearts by AP already matched Seihai (#55); no change.

Method (no traces kept here)

Real 2.22 client under Wine in a private network namespace against the lab GServer and list server; server-to-client messages injected by a re-encoding proxy; client messages read from the proxy's decoded log. Hearts also read from screenshots. No public server, no credentials.

`client-internals/ledges-push-pull.md`

Ledge jumps, pushing and pulling (2.05)

Date: 2026-10-28. Read from the 2.05 rev 2 client; tile lists from the 2.22 install's `arrays.dat`. **[2.05]** unless tagged. No addresses, routine names or decompiler text. Units are tiles; a tick is the 50 ms game tick, and a walk step is one arrow key in one tick. Test level: `new3.nw` (Seihai `client/tests/fixtures/levels/new3.nw`).

Summary

1. Running into a wall shows the `push` gani once the player has been stopped for 18 steps in a row (0.9 s).
2. Running into a ledge jumps the player off after the same 18 steps, with `jump.wav`: 8 ticks along a fixed curve. No gani changes in the air (the walk gani stays); keys are not read.
3. Running into a stone block pushes it one tile on the 29th blocked step, then every 20 steps. The player follows one tile; `stonemove.wav`.
4. A on a wall (nothing to lift) grabs it (`grab` gani) and holds while A is held. The arrow away from the wall plays `pull`. Every 18 ticks the client tries a pull. A pullable block comes one tile and the player steps back (`lift.wav`, or `lift2.wav` by a player flag not traced). Anything else, including a wall or a rock too heavy for the glove, does not move: the pull gani keeps playing and the `pulled` trigger still fires every 18 ticks. Letting go of the arrow returns to holding; letting go of A ends the grab.
5. Every push and pull try fires a trigger action (`pushed` / `pulled`) at a point in front, which NPCs there see as `actionpushed` / `actionpulled`; online it also goes to the server (PLI_TRIGGERACTION, 38) and to UDP peers (id 26).

Player modes involved

mode	meaning
0	normal
1	pushing (last step refused)
2	ledge jump (8 ticks)
4	lifting (5 ticks, then carrying)
5	grab / pull

The blocked-step count and the grab's pull cycle share one per-player counter.

The ledge test

The client counts blocked steps. A step counts when it is refused and no corner nudge was applied; a free step or a nudge sets the count back to 0, and so does the first refused step after a free one.

- At count 18 the ledge test runs. At 38 the count goes back to 18, so it runs again every 20 steps.
- Test point, from the player's position, by facing (up, left, down, right): `(1.5, 0.5)`, `(0, 2)`, `(1.5, 3.5)`, `(3, 2)`. The tile there must be a ledge tile **for that facing**. In the usual tile mode those are the four lists of `arrays.dat` array 8 (2.05 calls the class tile type 21, "jumping stone"):

facing	ledge tiles (2.22 `arrays.dat`)
up	190, 191, 4045, 4046
left	212, 228, 563, 579, 4028, 4044
down	185, 186, 4061, 4062

facing	ledge tiles (2.22 `arrays.dat`)
right	273, 289, 4031, 4047

- Landing point, from `(x + 1.5, y + 2)`: `(0, -5)`, `(-6.5, 3)`, `(0, 7)`, `(6.5, 3)`. It must pass the ordinary walking test (not a wall, not a chest). If it does not, the player just keeps pushing.
- The jump starts from the position after the refused step. Per tick `i = 0..7` (the first in the same tick as the start):

facing	position
up	`y0 - [1.5, 2.7, 3.8, 4.5, 5.0, 5.2, 5.2, 5.0][i]`
down	`y0 + [-0.2, 0.2, 0.5, 1.5, 2.8, 4.0, 5.4, 7.0][i]`
left / right	`x0 -/+ [1, 2, 3, 3.8, 4.5, 5.2, 5.8, 6.5][i]`, `y0 + [-0.2, -0.5, 0, 0.5, 1.1, 1.7, 2.3, 3.0][i]`

- `sounds/jump.wav` plays at `(x + 1.5, y + 2)` when it starts. Keys are not read until it lands.

In `new3.nw` (decoded with the 2.22 lists): the up ledges are the cliff edge at y = 14 and 15 (x 18 to 46); the left ledges are the two slanted cliff sides, x 14 to 16 and x 48 to 56, between y = 27 and 50. There are no down or right ledges. The push block is at 34,19, the pull block at 31,24, and 42,26 to 43,28 holds a mix of push and pull pieces that neither moves cleanly.

Pushing

At count 28 (checked before the step is counted, so on the 29th blocked step), and every 20 after, the client tries a push:

1. It fires the trigger action `pushed` at the test point above, with the facing as its parameter. NPCs there get `actionpushed`. Online the 2.05 client also sends it to the server and to its peers.
2. It reads the tile under that point. If it is one of the four tiles of the pushable block (`arrays.dat` array 16, entry 0: 1764, 1765 / 1780, 1781), that tile's place in the block gives the block's top-left. Only that one tile is compared, as in 2.05.
3. The two cells the block moves into must not be a wall, a chest, water, a chair or the lower half of a bed. From the block's top-left: up `(0,-1) (1,-1)`, left `(-1,0) (-1,1)`, down `(0,2) (1,2)`, right `(2,0) (2,1)`.
4. The block moves one tile. Each of its old cells first takes the tile two cells ahead, then the block is written one cell ahead. So the row the block leaves gets the floor that was in front of it.
5. The 2.05 client sends the 4 x 4 area around the old spot as a board modify, moves the player one tile in the facing, and plays `sounds/stonemove.wav`.

Pushed and pulled NPCs

When no block is found, the same routine looks for an NPC at the probe point.

- **Lookup.** The level's NPC hit test at the point. A character NPC is hit in `x + 0.5 .. x + 2.5`, `y + 1 .. y + 3`; a `setshape2` NPC by its shape's tile types; a GANI NPC by its animation; any other by its image box `[x, x + w/16) x [y, y + h/16)`, where a transparent pixel is a miss for plain image NPCs. The NPC needs `canbepushed` (GS1 command 44, cleared by 45). **[2.05]**
- **Edge.** The NPC is left out of its own test while every point along the edge it moves into is checked, one per tile from its left or top, `t` while `t < x + w` (or `y + h`): up `(t, y - 1)`, left `(x - 1, t)`, down `(t, y + h)`, right `(x + w, t)`. Sizes are in pixels / 16; character and GANI NPCs count as 48 x 48. Each

point goes through the client's blocked-point test: other players (online only), blocking NPCs, two level object lists (most likely dropped objects and chests), and wall tiles. Water does not stop an NPC. **[2.05]**
- **Move.** The NPC moves exactly one tile, the player one tile the same way, then `waspushed` fires on the NPC. `actionpushed` has already fired. The routine reports success, so the same sound as for a block plays. A character NPC that crosses a water edge gets the splash and switches between `swim` and `idle`. **[2.05]**
- **Server.** Nothing is sent for the NPC move: the NPC's x and y setters only store the value. The `pushed` trigger action is the only message. **[2.05]**
- 2.22 is the same, but its NPC lookup also searches the neighbouring map parts on a gmap. **[2.22]**

The trigger action on the wire

The `pushed` / `pulled` try goes to the server as `PLI_TRIGGERACTION` (38): GINT 0 (three spaces, no NPC), `GCHAR x*2`, `GCHAR y*2`, then the action name, and `,` plus the parameter when there is one, the direction as a decimal string (`pushed,0`). 2.05 also sends it to UDP peers (id 26). **[2.05]**

Grabbing and pulling

- **Order on A**: throw what is carried; lift; grab.
- **Grab test** at `(1.5, 0.9)`, `(0, 2)`, `(1.5, 3.1)`, `(3, 2)` by facing: a wall tile there, or an NPC that blocks. Nothing is grabbed when another player stands there.
- The grab lasts while A is held. Arrow keys do not move the player or turn them.
- **The pull cycle.** Holding the arrow away from the wall sets a counter to 2 and starts the pull pose. Each tick the counter goes 2, 3, ... 19, then back to 2 (18 ticks). Whenever the arrow is held at 19 the client tries a pull. Letting go of any arrow puts the counter back to 1 (just holding).
- **A pull try** fires `pulled` one tile out from the body (`(1.5, 0)`, `(-0.5, 2)`, `(1.5, 4)`, `(3.5, 2)`), with the opposite of the facing as its parameter, then reads the tile at the grab point. If it is a tile of the pullable block (array 16, entry 1: 1998, 1999 / 2014, 2015), the two cells behind the player must be free (from the block's top-left: up `(0,4) (1,4)`, left `(4,0) (4,1)`, down `(0,-3) (1,-3)`, right `(-3,0) (-3,1)`). NPCs with `canbepulled` are moved the same way and get `waspulled`. Then the block moves one tile towards the player (each old cell first takes the tile two cells behind), the player steps back one tile, and `lift.wav` plays (or `lift2.wav`, by a player flag that was not traced).
- **Pulled NPCs.** With no pullable block at the grab point, an NPC there with `canbepulled` (command 46, cleared by 47) is pulled the same way as a pushed one. First the two cells behind the player must be free, from the whole-tile grab cell: up `(0,3) (1,3)`, left `(3,0) (3,1)`, down `(0,-3) (1,-3)`, right `(-3,0) (-3,1)`. Then the edge facing the player is checked: up `(t, y + h)`, left `(x + w, t)`, down `(t, y - 1)`, right `(x - 1, t)`. The NPC and the player move one tile back and `waspulled` fires. Nothing goes to the server for the NPC. **[2.05]**
- **The futile pull.** On anything else (a wall, a rock too heavy for the glove, a push block, the mixed block pieces at 42,26 in `new3.nw`) nothing moves. The `pulled` trigger still fires every 18 ticks, and the pull pose keeps playing.

Sprites on the wire

From the animation step table (`decomp-findings-2026-10-24/sprite-step-table.tsv`): `push` is steps 14 + frame (S 11, 12, 13, 11, 12), `grab` step 19 (S 14), `pull` steps 20 + frame (S 15, 16, 17). The jump sends no sprite of its own.

arrays.dat

- **Array 8** holds four lists, one per facing (up, left, down, right): the ledge tiles. This is what tile type 21 ("jumping stone") means in the usual tile mode: the test asks whether the tile is in the list for the player's current facing. In the alternative tile mode (a tiledefs sheet of type 1 or 2) type 21 is instead the tiles in rows 30 and 31, columns 8 to 15, of the 16-column sheet.
- **Array 16** holds two 2 x 2 blocks, `[column][row]`: entry 0 the pushable block (1764, 1780 / 1765, 1781), entry 1 the pullable block (1998, 2014 / 1999, 2015). **[2.22]**

2.22 and 2.31

Both behave as 2.05 for everything above: the same counts, probe points, flight curve, block tiles, triggers and pull cycle (the number tables are identical in all three). The differences:

- **Grab on NPCs.** 2.05 grabs a blocking NPC only when a per-NPC flag is clear (most likely the character-shape flag). 2.22 and 2.31 grab any NPC that blocks there, and the lookup also checks the neighbouring map parts. The player and wall checks are unchanged. **[2.22] [2.31]**
- **Stop gates.** Walking, the ledge test, the A key and other gameplay code do nothing while either of two server-set flags is on. Server packets 176 and 177 set them; most likely these are the full-stop packets. **[2.22] [2.31]**
- **Sounds and custom ganis.** `jump.wav`, `stonemove.wav`, the pull sound and footsteps play only while the current gani is one of the built-in names (idle, walk, ..., push, grab, pull). The jump, push and pull still happen with a custom gani; it is expected to bring its own sounds. The sound names are also given bare (`jump.wav`) instead of under `sounds/`. **[2.22] [2.31]**
- **No jump gani** in either version: the flight sets none, so the walk or push gani stays. **[2.22] [2.31]**
- A further flag on the server connection also stops gani changes, footsteps and the A key; what it means was not found. **[2.22]**

Open

- ~~What is drawn right after A is let go while `pull` is playing.~~ Seen live (2026-10-30, `player-mannerisms-live.md`): `pull` keeps looping until the next key moves the player.
- Releasing A or the arrow mid-pull in 2.22 / 2.31 was not checked step by step.

`client-internals/level-editor-205.md`

Built-in level editor (client 2.05 rev 2)

Date: 2026-10-21

Derived from the 2.05 rev 2 binary: class names (RTTI), form resources (DFM), string tables, and the behaviour of the editor handlers. Names only, no code. Tags: **[2.05]** read from the binary · **[inf]** inferred.

This note covers 2.05. The primary editor reference is now `tools/level-editor/222-reference.md` (2.22), with the other builds as deltas in `tools/level-editor/version-deltas.md`.

In 2.05 the editor is not a separate program. It lives in the main window (`TAdventure`) and is a second mode of the game screen. GraalEditor.exe split off later; see `tools/graaleditor-vs-205.md`.

Entering and leaving

Key	Game mode	Notes
F4	open the editor	Refused when connected: "You cannot open the editor in the online mode." **[2.05]**
F5	Load level dialog (`TLoadLevel`)	Asks first when online: loading a level disconnects you.
P / Play button	back to game mode on the current level	A named, modified level is saved first ("Level saved"). "Please save the level under a new name before playing!" if it has no name.
Close button	close the current level tab	Asks "Save changes to …" when modified.

Entering the editor tears down the game state for the current level: looped sounds stop, NPC and baddy instances are freed, and the level the player stood on is opened as a tab (re-read from disk, so unsaved runtime board changes are not carried over). The scroll position centres on the player. **[2.05]**

Starting the client with a level file as the first command-line argument (`Graal.exe path\level.graal`) skips the login screen. The client changes into the file's folder, creates a local player at (30, 30.5), and opens the file in the editor. **[2.05]**

Window layout

- **Board view**: the DirectDraw surface shows the 64×64 level at 16 px per tile, with scroll bars.
- **Toolbar**: New, Open, Save, Save as, *Undo board copy*, Cut, Copy, Paste, Delete, *Turn selection into a warp field*, Edit links, Edit signs, Edit flags, Close, Play.
- **Tab bar**: one tab per open level. Each tab keeps its own scroll position, selection, floating paste, undo stack, and object lists. New levels are titled `(new)`.
- **Side panel**, with three radio modes:
    1. *Select predefined object*: stamps from GOBJSET01 sets (see below).
    2. *Select tile*: the tileset palette (`pics1.png`). Click to pick, drag to pick a block. Double-click a tile to make it the background tile shown in the preview box.
    3. *Baddies, Npcs, Chests*: a palette of placeable objects.
- Checkboxes: *Show blocking tiles*, *Show foreground tiles*, *Show terrain tiles* (overlays computed from the tile-type table, not stored in the level).
- Status bar: tile coordinates, tile index, and the object under the cursor (`tile`, `npc`, `chest`).

Editor keys

The editor uses Delphi shift states. Ctrl is the main modifier.

Key	Action
Ctrl+N	New level
Ctrl+O	Open
Ctrl+S	Save
Ctrl+Z, Alt+Backspace	Undo board copy
Ctrl+X, Shift+Del	Cut
Ctrl+C, Ctrl+Ins	Copy
Ctrl+V, Shift+Ins	Paste
Del	Delete selection
L	Edit links

Key	Action
S	Edit signs
P	Play
M	Generate a map image (`graalmap.png`, "Generating the map may take some minutes. Proceed?"). It walks the level list of the loaded big map (`bigmap.txt` or the list a `setmap` NPC loaded), 56×56 px per level, and does not follow links.

L, S, P and M are ignored while a combo box in the side panel has focus.

Tile editing model

- **Selection.** Dragging on the board selects a tile rectangle. The selection is clipped to the 64×64 board.
- **Floating paste.** Copy or Paste lifts the selected tiles into a floating block that follows the mouse. Clicking outside it drops it. Dropping writes the tiles onto the board. Moving a selection works the same way: it is lifted, dragged, and dropped.
- **Right mouse button** on a placed baddy, NPC, or chest makes a copy of it ("Right Mousebutton-> copy …").
- **Warp field.** *Turn selection into a warp field* creates a link whose rectangle is the selection and opens the link dialog.

Undo

Undo covers **board tiles only**, which is why the button says *Undo board copy*.

- Before any write to the board, the editor stores an undo record: the rectangle (x, y, width, height) plus the tile values it is about to overwrite.
- Records are kept per level tab in a stack of at most **128**. The oldest record is dropped when the stack is full.
- Undo pops the newest record, writes the old tiles back, re-selects that rectangle (or clears the selection if it was the whole board), and marks the level modified.
- Adding, moving, or deleting links, signs, NPCs, baddies, and chests is **not** undoable.

Object types

The editor's level document (`TAdventureLevel`) keeps separate lists, one per kind.

Kind	Class	Placed with	Dialog (double-click)
Link	`TEditorLink`	warp-field button, or the link list	`TLinkList` → `TLinkOptions`
Sign	`TEditorSign`	sign list "New"	`TEditSigns`
Chest	`TEditorChest`	object palette	`TChestEdit`
NPC	`TEditorNPC`	object palette	`TEditPerson` → `TPersonAction`
Character NPC	`TEditorNPC` with a generated script	object palette	`TEditCharacter`
Baddy	`TComputer`	object palette	`TEditBaddy`
Stamp	`TEditorObject`	"Add the selection to the objects"	`TObjectName`, `TObjectSet`

Links (`TLinkList`, `TLinkOptions`)

A list with columns Linkfile, X, Y, W, H, New X, New Y, and buttons Edit, Delete, Load. *Load* opens the destination level in a new tab. The link dialog accepts a file name or URL, and the new position may be an expression: "you can use math. operations and variables like playerx/y". The tutorial levels use `playerx` and `playery` this way.

Signs (`TEditSigns`, `TShowCodes`)

A list of signs with X and Y fields, New, and Delete. *Show special codes* lists the glyph escapes (`#h` head, `#A`, `#B`, …). A sign at (0, 0) is not drawn on the board. Such a sign is used by index: a chest shows sign *n* when opened, and the GS1 `say n` command shows sign *n*. The embedded tutorial uses both.

Chests (`TChestEdit`)

The chest holds an item and a sign index (-1 = no sign). The 2.05 item list, in index order:

```
greenrupee, bluerupee, redrupee, bombs, darts, heart, glove1, bow, bomb, shield, sword,
fullheart, superbomb, battleaxe, goldensword, mirrorshield, glove2, lizardshield,
lizardsword, goldrupee, fireball, fireblast, nukeshot, joltbomb, spinattack
```

A new chest defaults to item 1 (bluerupee) and sign -1.

Baddies (`TEditBaddy`)

There are 10 types: Gray Soldier, Blue Soldier, Red Soldier, Shooting Soldier, Swamp Soldier, Frog, Spider, Golden Warrior, Lizardon, Dragon. The script names are `graysoldier` … `dragon`, and the sprites are `baddy*.gif`. Each baddy has three verses: *attack*, *when hurt*, and *when it hurts the player*. A new baddy starts with "Stop!" and "Ouch!". Types 4–6 (Swamp Soldier, Frog, Spider) do not talk, and the writer saves empty verses for them.

NPCs (`TEditPerson`, `TPersonAction`)

An NPC has a GIF image (or none) and a GS1 script. The script editor has *Test* (syntax check, with the error shown in a status line) and *Style*, plus the usual Undo/Cut/Copy/Paste menu. Runtime flags set by scripts while testing appear in *Edit flags* (`TEditFlags`). That dialog edits the in-memory flag list, not anything stored in the level. The list survives Play and a return to the editor, but it is never written to disk and is lost on exit (live).

Character NPCs (`TEditCharacter`)

A form generates a script that sets up a walking character. The fields are character, shield, nickname, horse, direction, walk mode (None/Random/Left-Right/Up-Down/Rectangle), speed, attack mode (None/Sword), hunt speed, sword power, attack verse, power, roaming box X1–X2/Y1–Y2, respawn time, and the flags *can be carried*, *sword hurts*, *bomb/arrow/stone hurts*, and *attack other npcs too*. The output starts with `// Graal v1.4 NPC by Stefan Knorr` and `showcharacter;` / `setcharprop` lines. After that the NPC is an ordinary NPC: double-clicking it opens the normal NPC editor. **[2.05]** Several tutorial NPCs were made this way.

Stamps (predefined objects)

Object sets are read from every `objects*.txt` file listed in the file-name cache. The set name is the part between `objects` and `.txt`, and plain `objects.txt` is the *Standard* set. Each file is `GOBJSET01` (see `formats/gobjset01.md`). *Add the selection to the objects* names the current selection (`TObjectName`) and inserts it after the selected stamp. *New* (`TObjectSet`) creates `objects<setname>.txt`. *Save objects* rewrites the set file.

Saving

The file extension picks the format. **[2.05]**

- **`.nw`**: GLEVNW01 text. It writes `GLEVNW01`, then 64 `BOARD 0 y 64 0 <base64 tiles>` rows, then `LINK …`, `SIGN x y …` `SIGNEND`, `CHEST …`, `BADDY x y type` + three verse lines + `BADDYEND`, and `NPC image x y …` `NPCEND` (image `-` when there is none).
- **Anything else** (normally `.graal`): always `GR-V1.03`. The 2.05 editor cannot write older GR versions.

GR-V1.03 writer details, which complement `formats/level-binary.md`:

- The board is written with 13-bit codes. The encoder emits a single run (`count` ≤ 64) for two or more equal tiles, and a **double run** (`ABAB…`, ≤ 64 pairs) when the pattern `A B A B` starts at the cursor. Otherwise it emits a plain tile. So 2.05 does write double runs, which answers one of the open questions in that file.
- Baddy records are `x, y, type` as raw bytes (not +32), then `verse1\verse2\verse3\n`. The section ends with `FF FF FF 0A`.
- NPC records are `x+32, y+32, image#script\n`. Chest records are four GCHARs. Sign records are two GCHARs plus sign-alphabet text.
- Saving a level that was opened from a URL (`http://…`) falls through to *Save as*.

Opening accepts `Z3-V1.03`, `Z3-V1.04`, `GR-V1.00`–`GR-V1.03`, and `GLEVNW01`. Anything else gives "Unsupported file type!".

Loader note for formats/level-binary.md

The 2.05 loader picks the code width by magic: **13 bits only for `GR-V1.02` and `GR-V1.03`, 12 bits for `GR-V1.00`, `GR-V1.01`, and `Z3-V1.*`.** The section order also depends on the magic: NPCs from `GR-V1.00` on, and chests from `GR-V1.01` on. The baddy section ends at the first record shorter than three bytes or whose first byte is ≥ 0x80. It does not need a literal `FF FF FF`. An x byte of 0 is a valid baddy, not a terminator (corrected 2026-10-25; the loader has no x = 0 test in 1.39, 2.00, 2.05, 2.17.1 or 2.22; see `decomp-findings-2026-10-25-f/03-baddy-terminator.md`). **[2.05]**

Open

- *Generate map* and *Edit flags* were answered live; see `live-checks-editor-offline-sound.md`.

`client-internals/live-behaviour-404.md`

Graal 4.04r2 live: the Login server, chat, movement, carrying and respawn

Date: 2026-10-30. The real 4.04r2 Windows client (token `G3D17026`) run under Wine 9 in an isolated, loopback-only lab, against the prebuilt GServer-v2 through a gen-5 logging proxy. No addresses, routine names or decompiler text.

Tags: **[live]** seen with the real client · **[bin]** read from the 4.04r2 engine · **[src]** GServer-v2 source · **[inf]** inferred.

Caveats. The lab level is `onlinestartlocal.nw` from GServer-v2's default world. The ganis, bodies and item art come from the lab server (2.22 `graal_reborn` files), so any look that is drawn by a gani is the server's, not the client's. Respawn timing is GServer-v2's (`respawntime = 15`).

The Login server never draws players

Logging in to `<login-host-4x>:14900` opens a session to a server that the client names `Login`. The level it receives is only a backdrop for the server-list window. **[live][bin]**

- When the engine sets up the player and level, it sets the draw-layer mask to 0 if the lowercase server name starts with `login`, and to all layers (0x7ff) otherwise. With mask 0 no players and no HUD are drawn. Ghost mode (S→C 170) forces its own mask (0x500) separately. **[bin]**
- The Login server hands the client over with **S→C 178 (`PLO_SERVERWARP`)**: comma text `x,<name>,<host>,<port>`. Field 0 is not used, and fewer than 4 fields are ignored, as in 2.31 (`protocol/packet-ids.md`). The client closes the Login session, logs in again to that host and port, and uses `<name>` as the server name, so the game server must not be named `Login`…. **[live]**
- In the lab, injecting `178` after the Login session's 156 made the player, the HUD (A/S/D, AP bar, hearts) and the player list appear. Before that, everything else that was tried changed nothing. That included dropping GServer-v2's id 195 answer to 157, adding `GANI0001` headers, and placing body and sprite images locally. A separate lab run found the same on the 5.00r7 client. **[live]**
- For a server emulator: either send 178 from the login session, or give the session a server name that does not start with `login`. GServer-v2 sends 178 only when its listserver forwards a server-info request.

Other login details seen live: GServer-v2 reports the client as `G3D17026 (4.042)`. The client asks for `basepackage.gupd` and `bigmap.txt` first. It answers S→C 182 with id 44 (its window titles). The saved password field was empty after a restart under Wine.

Chat

- **Tab** opens a dark red chat bar along the bottom of the game view, and Enter sends. Letter keys without Tab are game keys. **[live]**
- The text is drawn centred above the head in white bold sans, with a pure blue (#0000FF) copy 1 px down and right as a shadow. There is no bubble, frame or background, the same as the classic clients (`chat-text.md`). **[live]**
- Sent as C→S 2, prop 12 (`PLPROP_CURCHAT`). **[live]**



4.04r2 chat text over the player

Walking and pushing against a wall

- While a key is held, the client sends X2/Y2 (props 78/79) every 50 ms tick. The value is half-pixels, with the sign in the low bit. A walk step is 16 half-pixels, which is 8 px or half a tile per tick (10 tiles/s). **[live]**

- Against a wall the `push` gani starts about 0.9 s after the player stops moving, which matches 2.05's 18 blocked steps (`ledges-push-pull.md`). The client sends 157 `push` and prop 10 `push`, and goes back to `idle` when the key is let go. **[live]**
- While pushing, the client also sends a trigger action (C→S 38) `pushed,0` at a point in front of the player, about 0.55 s after the `push` gani starts and then about once a second. This is the online form of 2.05's `pushed` trigger. **[live]**
- A mouse click on the game view sends a trigger action `leftmouse` at the clicked point. **[live]**

Action key (A), sword (S), item (D)

- **A on something that cannot be lifted** (here, a green stone with the starting glove): `grab` gani (C→S 23 `grab.gani`, 157 `grab`, prop 10 `grab`). **[live]**
- **S**: C→S 34 with `sword.gani`, 157 `sword`, prop 10 `sword`, then `sword.wav`. Back to `idle` about 250 ms later. **[live]**

Cutting and lifting a bush, and respawn

- **Cut.** A sword hit on a bush sends **C→S 1 (`PLI_BOARDMODIFY`)** `x, y, 2, 2` with the four stump tiles, plus `crush.wav`. The server echoes it to the level as S→C 7. **[live]**
- **Lift.** A facing a bush sends the same 2×2 board modify, then `lift` (gani, 157, prop 10) with `item.wav` and `lift.wav`. About 300 ms later the gani becomes `carrystill`, and prop 19 (carry sprite) becomes 1 (bush). GServer-v2 then dropped an item (S→C 22, here a rupee). **[live]**
- **Respawn is the server's.** The client keeps no timer: the bush comes back when the server sends S→C 7 with the original tiles. GServer-v2 did that 14.5 s after a cut and 14.2 s after a lift, from its 15 s `respawntime` checked once a second. **[live][src]** For what the original servers did, see `board-respawn.md`.

Carrying, hiding and throwing

- **Carrying** sends `carry` while walking and `carrystill` while standing. The held object is the gani's `ATTR3` sprite, drawn last and 22 px above the body. The head and the raised hands are drawn first, so the object covers the top of the head. That comes from the gani, so another server's ganis can draw it differently. **[live]**

- **D while carrying hides the player under the object**, as in 2.05 (`carry-hide-and-glove.md`). The client sends `hiddenstill`, then `hidden` while walking, and sets status (prop 18) to 0x16, which has bit 0x02. Only the bush is drawn, and the player is not. Hidden walking moves 1.6 px per tick on average (steps of 4 and 2 half-pixels), which is 0.2 of a normal step, the same factor as 2.05. **[live]**



- **A while carrying throws**: C→S 11 (`PLI_THROWCARRIED`) with an empty payload, then `idle`, and the carry sprite is cleared. **[live]**

Pots, stones and black stones

For these, two pots, a stone and a black stone were placed on the lab level, and the lab account was given glove power 3. The tile numbers are the ones in `board-respawn.md`. **[live]**

Object	Glove needed	Board after lifting	Carry sprite (prop 19)	Item drop (GServer-v2)	D hides	Thrown	Back after
Bush (2)	1	677 678 / 693 694 (stump)	1	sometimes (a rupee seen)	yes	leaves, `crush.wav`	14.2 to 14.5 s
Pot (684)	1	1770 1771 / 1786 1787 (empty base)	5	a heart, every time	**no**	shards, `crush.wav`	15.0 s
Stone (34)	2	1834 1835 / 1850 1851 (hole)	3	none	yes	fragments, `crush.wav`	15.0 s
Black stone (990)	3	1834 1835 / 1850 1851	GCHAR byte 0xe9 (201), not decoded	none	yes	`crush.wav`	14.3 s

- With glove 1, A on a stone plays `grab` instead, as in 2.05. **[live]**
- A pot gives `item.wav` with the lift, because the server drops its heart at the same moment. Stones only give `lift.wav`. **[live]**
- The thrown object breaks about 250 to 350 ms after the throw, which is when `crush.wav` is sent. No board change or other packet marks where it landed. **[live]**
- Coming out of hiding with A sends `carrystill` and status 0x14, which clears bit 0x02. A second A throws. S throws straight from hiding. **[live]**
- Every respawn is a single S→C 7 with the original 2×2 tiles. The 14 to 15 s spread is GServer-v2's 15 s timer, checked once a second. **[live][src]**



pot: lift, D (no hide), throw and shards



black stone: carried, hidden (D), thrown, gone

Hidden state survives a reconnect

The client was closed while the player was hidden. GServer-v2 saved `ANI hiddenstill` and `STATUS 22` to the account, but nothing was being carried. On the next login only the shadow was drawn, and the player came back on the first step. A server should clear the hidden status bit and the hiding gani when it saves or loads an account. **[live]**

4.20r8 (the packet-logger build)

The 4.20r8 engine survives only in a third-party packet-logger build (`protocol/v4x-clients.md`). Its logging hook calls fixed Windows XP SP2 addresses, so it crashes anywhere else. In a local copy, the eight original bytes at the hook's only call site (the end of zlib's inflate) were put back, which turns the build into a plain 4.20r8 engine. That copy ran under Wine with the 4.04r2 data files. **[live][bin]**

- GServer-v2 accepts it with token `G3D20126`, and it goes through the same Login server and S→C 178 handover. **[live]**
- Chat: the same white bold text with a blue shadow, and the same dark red Tab bar. After Enter, the bar stayed open with the sent text in it. **[live]**
- Walking is 8 px per tick. The `push` gani starts 0.85 s after the last step, and the `pushed` trigger action follows 0.55 s later and then once a second, as in 4.04r2. **[live]**

Live run 2026-10-31: 4.04r2 and 4.20r8 next to Seihai

A second lab, run in a cloud container: Wine 9, Xvfb, a loopback-only net and mount namespace, the seihai-lab server pack with stock gs2lib, and a proxy on 14900 that injects S→C 178 `0,Lab Server,<loopback>,14900`. The 4.20r8 engine was restored from the packet-logger copy by putting back the 8 bytes at its hook: `GraalEngine.dll` offset 0x1a3ae3, `e9 f3 cc 08 00 90 90 90` back to `5f 5e 5d b8 01 00 00 00`. **[live] [bin]**

- **After the login, both builds send:** 37 (language); 23 `basepackage.gupd` twice; 23 `bigmap.txt` (in the Login session only); 157 `idle`; 152 `GraalEngine,pmguilds,""` and `GraalEngine,pmservers,""`; 154 `GraalEngine,lister,options,globalpms=true,buddytracking=true,showbuddies=true` and `GraalEngine,lister,verifybuddies,1,0`; **46**; 44 (window titles); then 2 (own props). GServer-v2 logs the 152 as RequestText and the 154 as SendText. **[live]**
- **Walking** is the same in both builds: 2 with GANI `walk` and SPRITE on the first step, then X2 (78) or Y2 (79) alone every 50 ms, then `idle`. X/Y (15/16) are never sent. **[live]**
- **Script flags.** A GS1 probe weapon ran these lines on `playerenters`:
  - `setplayerprop #c`
  - `setstring gr.probe,1`
  - `setstring client.probe,2`
  - `set probeflag`

  The 4.04r2 client sent only C→S 18 `client.probe=2`. The lab's `-gr_movement` weapon (`setstring gr.x` every 0.05 s) sent nothing either. So the client runs GS1 weapon scripts but sends only `client.` string flags to the server. **[live]**
- **Chat bar.** The chat bar stays open after Enter in 4.04r2 as well. Tab closes it and sends an empty prop 12. Esc opens "Do you really want to quit Graal?". **[live]**
- **Bow.** D with the bow sends 157 `shoot`, prop 10 `shoot,wbow1.png`, `arrow.wav`, and C→S 9 (arrow). With bombs, D sends C→S 4. **[live]**
- **In the Login session the game works.** Walking, chat and the sword all send as normal. Only the drawing is cut, as described above. **[live]**
- **Ids.** C→S ids seen: 2, 4, 9, 23, 24, 34, 37, 38, 44, 46, 152, 154 and 157. S→C ids: the 26 from the first run plus 45. **[live]**

Seihai's write-up, with the fixes this run led to (X2/Y2 walk, `client.` flags only), is in seihai `documentation/v4-profiles.md`. The logs are in the project folder under `v4/lab-4x-live-2026-10-31/`.

Third run, 2026-10-31: update packages, frames, focus and the walk in detail

The same lab, rebuilt in a fresh container (stock 4.04r2, seihai-lab pack with stock gs2lib, `warpproxy.py` on 14900). Logs are in the project folder under `v4/lab-4x-after-login-2026-10-31/`. **[live]**

- **Update packages.**
  - With no copy, the client asks `basepackage.gupd` with C→S 23, twice.
  - With the server serving it, the client saves it in `updatepackages/basepackage.gupd`. On the game server it also saves a copy in `updatepackages/<server name, space as %032>/` (`Lab%032Server`).
  - On the next run of the program it sends **C→S 47 (`VERIFYWANTSEND`)** twice instead: a GUINT5 CRC-32 of the cached file, then the name. For `GRPKG001` and a newline that is `! MCt'basepackage.gupd`, CRC 0x15a8ea07.
  - Within one run it keeps asking with 23 (its in-memory copy is not checked). This is the same rule 5.00r7 follows (`#145`).
- **`bigmap.txt`** is asked once per run of the program. It is asked on the Login server and not after the warp.
- **The after-burst messages** come in four frames a few milliseconds apart:
  - 23 `idle.gani` and 157 `idle`
  - the head image, 152 ×2 and 154 ×2
  - 46
  - 44, the default weapons' images, 2 (own props), 24, then 34 `pics1.png`

  157 is `PLI_UPDATEGANI`: a GUINT5 checksum (0, since the client has no copy), then the name. It goes with each 23 for a GANI (`idle`, `walk`, `sword`, `shoot`), once per connection.
- **Nickname.** Prop 0 in the own props is the start screen's Nickname field (`Real404`), not the account.
- **Walk.**
  - The first step sends `GANI walk` and SPRITE, then the moving axis (X2 or Y2).
  - Each 50 ms tick sends only the axes that moved: both X2 and Y2 on a diagonal (`n*@o)@`), and only the first axis in the very first message.
  - `idle` goes out alone one tick (50 ms) after the last step.
  - Every step is 8 px.
- **Focus.** Losing the window focus sends C→S 2 STATUS 21. Getting it back sends 20. No position goes with either. **[live]**
- **24** is `GCHAR 9` every 5.142 to 5.151 s while alone in the level. A tick can carry it in a walk frame.

Seihai's 4.x profiles now send all of the above except the GUI image checks (squirelworks/seihai#295).

Not covered yet

- The client's own ganis, from a 4.x-era server rather than the 2.22 world, to compare the look.
- Signs (lift entry 1).
- Carrying and hiding on 4.20r8. Only chat, walking, pushing, the sword and items were repeated there.

`client-internals/live-checks-editor-offline-sound.md`

Live checks: editor, offline mode and sound (2.05, 2.22, 2.31, GraalEditor)

Date: 2026-10-30

These results come from running the real clients in the Wine lab described in `live-trace-205.md`: Wine 9 with a 32-bit prefix, Xvfb, a network namespace with only loopback, and an ALSA `null` device so that DirectSound starts. File access was logged with Wine's `+file` channel, and which file each action used was read from that log. The code paths named below come from the 2.05 rev 2 decompile. Names only, no code.

Tags: **[live]** seen in a running client · **[2.05]** read from the 2.05 binary · **[inf]** inferred.

Corrections to earlier notes are collected at the end.

Offline mode

The Training button

All three clients tested have a **Training** button on the start screen. 2.05 rev 2's welcome text says: "press 'Training' for some offline training levels, and 'Start' for the online mode." Pressing it starts offline play on the embedded tutorial and makes no network connection. **[live]**

Client	Start screen buttons	Training starts on	Big map / minimap
2.05 rev 2	Training, Options, Start	`tutorial_b-4.graal`: the welcome sign, Stefan, the two coloured lights	`tutorialmap.txt` set by the level's `setmap` NPC
2.22	Training, Options, Get an account!, Start	`offlinetutorial_b-2.graal`, the centre of the 3×3 grid: wizard, cauldron, two torches	the client's `bigmap.txt` / `minimap.txt`, plus `maps/parts/offlinetutorial/offlinetutorial_*.graal.png`
2.31	Training, **Editor**, Options, Get an account!, Start	`offlinetutorial_b-2.graal`, the same as 2.22	the same as 2.22

The 2.22 and 2.31 start level plays `cauldron_bubbles.wav` and `fire_torch0..2.wav` as soon as it loads. Those sounds come from its NPCs. **[live]**

The embedded level always wins

A file on disk with the same name does not replace the embedded level. **[live]**

- **2.05**: with `tutorial_b-4.graal` (a different level's contents) in the game folder, Training still loads the embedded b-4. The client does not even look for the name on disk.
- **2.22**: with `offlinetutorial_b-2.graal` in the game folder, the client looks the name up once but still loads the embedded copy.

Tile definitions when offline

Every client tested reads `levels/tiledefs/tiledefsOffline.txt` when Training starts. That includes 2.05, which builds the file name as `tiledefs` + the server name + `.txt`, and the server name offline is `Offline`. The shipped file has one line, `picso.png,offlinetutorial,1,0,0`, which applies to the `offlinetutorial*` levels only. So in 2.05 the read happens but has no effect on the old `tutorial_*` levels. **[live][2.05]**

The reconnect dialog and *Play offline*

Pressing *Start* in 2.05 with no listserver shows "The connection couldn't be established!" and stays on the server list. That screen has no *Play offline* button. **[live]**

The reconnect dialog has three buttons: *Reconnect*, *Open serverlist* and *Play offline*. It was reached two ways in the lab. **[live]**

How the connection ended	Dialog text	*Play offline* goes to
The listserver listed a server, but its game port refused the connection (TCP reset)	"The connection couldn't be established!"	`tutorial_b-4.graal` with the welcome sign, the same as Training. No level file was read from disk.
Logged in, then the game socket was destroyed from outside (`ss -K`)	"You have been disconnected from the game server."	The same level, reloaded from the game folder (`onlinestartlocal.nw`), with the player where they stood.

Two lab notes:

- Closing the server side normally (a FIN) was not enough. The client left the socket half-closed and showed no dialog. The dialog came up only after a reset or a socket error.
- For the second row, a copy of the online level was in the game folder. The case where the level is not on disk was not repeated after a drop. The code says it falls back to the tutorial (see `offline-tutorial-205.md`).

Built-in editor (2.05)

Generate map (M)

After the "Generating the map may take some minutes. Proceed?" prompt, the editor works through the level list of the **currently loaded big map**. That is the list from `bigmap.txt`, or whichever list a `setmap` NPC loaded (for the tutorial, `tutorialmap.txt`). **[live][2.05]**

- It reads each listed level from disk in list order, row by row, and loads it as a level. Then it runs the level's NPCs into a 1024×1024 view and shrinks that to **56×56 px** in the output image. Missing or unreadable entries become blank squares.
- It does not follow links. Levels that are not in the list are skipped, for example `tutorial_b-2_house.graal`.
- The result is offered in a save dialog as `graalmap.png`, defaulting to the level's folder. With the 3×4 tutorial list it produced a 168×224 PNG.
- Afterwards, the level that was open in the editor is restored.

Play saves first

Pressing *Play* on a named, modified level writes the level to disk ("Level saved" in the status bar) before the *Level options* dialog asks for the start position. **[live]** This matches the 2.22 behaviour in `tools/level-editor/222-reference.md`.

Edit flags

Edit flags edits the client's in-memory flag list. Flags typed into it are still there after Play and a return to the editor. Nothing is written when the dialog closes or when the client exits: the test flag was not in any file in the game folder or the Wine prefix, nor in the registry. The list does not survive a restart. **[live]**

2.31 and GraalEditor

- The 2.31 start screen's **Editor** button starts `GraalEditor.exe` from the game folder as a separate process. The 2.31 client still has no editor forms of its own. **[live]**

- GraalEditor 2.3.1.1 runs under Wine. It opens on a new `new2.nw` tab with the four palette modes, including *Predefined NPCs*. **[live]**
- At startup it looks for `arrays2.sig`, `arrays2.dat`, `graaleditor.sig`, and `arrays.dat`. The shipped folder has no `GraalEditor.sig`, and the editor starts normally anyway. With one byte of `arrays2.sig` flipped, it also starts normally, with the same file reads and no message. Whatever the RSA check controls, it does not stop the editor from starting. **[live]**

Sound (2.05)

With an audio device present, DirectSound starts and the client plays effects. Each effect shows up as a lookup of its file under `sounds\`, and that is how the rows below were matched to actions. Most rows were triggered in a test level: water, a pushable stone block, a bush, ledge rows, and chests. GS1 NPCs fired the remaining effects on staggered timers (`putbomb`, `putexplosion2`, `shoot*`, `putleaps`, `spyfire`, `lay`, `puthorse`, `putcomp`, `hurt`, `setgender`). **[live]** marks a sound heard in the running client. **[2.05]** marks a sound known only from the call sites of the sound routine.

Action	Sound	Source
Walking on ground	`steps2.wav` about every 0.2 s; `steps.wav` when a move starts	[live]
Swimming	`water.wav` about every 0.8 s	[live]
Riding a horse	`horse.wav` and `horse2.wav`, alternating about every 0.4 s	[live]
Sword swing	`sword.wav`; with the battle axe, `axe.wav`	[live]
Lifting a bush	`lift.wav` for a male player, `lift2.wav` for a female one (`setgender female`)	[live]
Getting hurt	the same `lift.wav` / `lift2.wav`: `hurt`, an explosion, a thrown ball or a baddy hit all play it	[live]
Dying	`dead.wav`	[live]
Laying a bomb	`put.wav`	[live]
Bomb or explosion	`bomb.wav` (`putbomb` of any power, `putexplosion`, `putexplosion2` power 2, the end of a fireblast or nuke)	[live]
Jolt explosion	`thunder.wav` (`putexplosion2` power 3)	[live]
Launching any projectile	`arrow.wav` for arrows, fireballs, fireblasts, nukes and `shootball`, and for arrows from shooting baddies and the dragon	[live]
Fire spy	`fire.wav` (`spyfire`)	[live]
A thrown object or leap landing	`crush.wav`; `water.wav` for the water leap (type 5)	[live]
Picking up a laid item (rupees, battle axe)	`extra.wav`	[live]
Opening a chest	`chest.wav`, also when the chest has a sign. No item sound follows.	[live]
Sign pages	`nextpage.wav` on every page turn	[live]
Pushing a stone block	`stonemove.wav` about once a second while it moves	[live]
Ledge jump	`jump.wav`	[live]
Frog	`frog.wav` when it hops	[live]
Baddy walking	`steps.wav`	[live]
Baddy dies	`compudead.wav`	[live]
Baddy draws its sword	`swordon.wav`. Six baddy types were tried; none played it.	[2.05]

Action	Sound	Source
An arrow hitting the player	`arrowon.wav`. Not reached: no test arrow hit the player.	[2.05]
`item.wav`, `item2.wav`	Only used in the sign-reading routine and in a server-message handler. The chest and laid-item tests didn't play them.	[2.05]
Another player's bomb or item (online)	`bomb.wav` / `thunder.wav`, `item.wav`, `extra.wav`, from server messages	[2.05][inf]
GANI `PLAYSOUND`, GS1 `play` / `playlooped`	the named file	[2.05]

The lift/hurt sound is picked by a single byte on the player object: `lift.wav` when it is set, `lift2.wav` when clear. New players start with it set. That `setgender female` clears it is inferred from the sound changing. **[2.05] [inf]**

Corrections to earlier notes

- `offline-tutorial-205.md` said there is no offline item on the start screen. There is: **Training**. It is the usual way in, and *Play offline* on the reconnect dialog is the other.
- `offline-tutorial-205.md` and `tools/graaleditor-vs-205.md` said only 2.22 reads `tiledefsOffline.txt`. 2.05 and 2.31 read it too (see above).
- `tools/graaleditor-vs-205.md` said 2.31 has no editor. It has no editor *forms*, but its start screen launches GraalEditor.
- `level-editor-205.md` listed Generate map and Edit flags as open. Both are answered above.
- The first version of the sound table in this note guessed that `lift2.wav` is for heavy objects and that `fire.wav` is the fireball. Both were wrong. `lift2.wav` is the female lift and hurt sound, every projectile launches with `arrow.wav`, and `fire.wav` belongs to `spyfire`.

`client-internals/live-gserver13.md`

The real 1.3 server and client, live under Wine (2026-10-30)

The 1999 `GServer13.exe` and the 1.3 test client `Graal13.exe` (both from the `graal13b` archive) were run together under Wine, inside a loopback-only network namespace. The traffic was captured with tcpdump. This is the first time an original server and its own client were played together in the lab. Tags: **[live]** seen in the running programs, **[cap]** read from the capture.

The raw capture, the decoded message list, screenshots, the run notes and the tools are in the project files under `graal-decompile/live-trace/early/`. This note has no addresses and no decompiler text.

Server settings and the default account [live]

GServer13 opens a start dialog before it listens:

setting	default
port	14804
respawn time (bushes etc.)	10 s
horse lifetime	10 s
alignment increment time	1200 s
baddies respawn time	60 s

These values are saved to `serverconfig.txt` as `serverport`, `respawntime`, `horselifetime`, `apincrementtime` and `baddyrespawntime`.

A new account from Edit accounts starts with these values:

- level `onlinestartlocal.graal` at x 30.5, y 30
- AP 50, MP 0
- 3 hearts of 3
- 0 rupees, 10 bombs, 5 arrows
- glove 1, bomb 1, bow 1
- sword 1 `sword1.gif`, shield 1 `shield1.gif`
- head `head0.gif`, colors orange, white, blue, red, black
- weapons enabled, no spin attack

The rights levels are no rights, Graalian Police, Trusted Admin and Administrator. The 1.3 zip does not ship `onlinestartlocal.graal`. It was copied from the 15th Graal build, which uses the same `GR-V1.03` level format.

Login [cap]

- **Client's first frame.** u16 big-endian length + zlib of id 0 + `GSERV014` + length-prefixed account + length-prefixed password + `\n`. Lengths are chr(n + 32).
- **Wrong account.** The server sends id 16 with the text "An account with the name and password you typed in doesn't exist.", and the client offers Reconnect or Play offline.
- **Good login, server order.**
    1. 9: player props.
    2. 14: warp, with x, y and the level name.
    3. 100: raw-data size, 8194.
    4. A **separate** zlib frame holding 101 (board) + 8192 bytes. In 2.05 the size message and the board share one frame.
    5. 6: level name.
    6. 1: level links.
    7. 5: signs.
    8. 39: level modification time.
    9. 3: NPC props, one message per NPC, carrying the whole script.
    10. 1.
- **Client's next messages.** It asks for each NPC image with 23, sets flags from the NPC scripts with 18, and adds horses with 7 (from `puthorse`). The server answers 30 (file not sent) for images it lacks. It deletes each horse with 18 about 9.5 to 9.8 s later, which matches the 10 s horse lifetime on a 1 s server tick.
- **Server sweeps.** The horse deletes and the bush regrows all landed on the same fractional second, so the server checks its timers once a second.

In play [live] [cap]

- **Walking.** While a key is held, the client sends one id 2 (player props) message every 100 ms, carrying X (prop 15) and SPRITE (17). Speed is 0.5 tile per 50 ms tick, the same as 2.05.
- **Chat.** Sent as id 2 prop 12. It is drawn as plain white text with a blue outline above the head, with **no bubble** (`shots/chat13.png`).
- **Cutting a bush.**
    - The client sends id 1 (board modify) with x, y, w = 2, h = 2 and the four leftover tiles.

    - The server does not echo the change back to the cutter.
    - The server regrows the bush with id 7 and the original tiles 2 3 / 18 19.
    - Two cuts regrew after 7.6 s and 9.2 s. Both fit the 6 to 12 s formula in `board-respawn.md`.
    - A bush cut also dropped rupees, which were rolled on the client, as that note describes.

Wine notes for 1.x clients

- **Keyboard.** The 1.3 client gives DirectInput a keyboard data format that lists one object, with a 256-byte buffer. Windows filled the whole buffer. Wine fills only the listed objects, so the arrow keys never reach the game. Keys do reach DirectInput, and chat and Alt shortcuts work, so the player just stands still.
    - Patching the exe on disk trips the client's own checksum ("The Graal execution file is corrupted").
    - `tools/kbfix.py` in the project files instead rewrites the format in memory to a full 256-key table right after start, before the client sets it. Walking then works.
    - Other Delphi builds that use the same DirectX header probably need the same fix at their own address.
- **Display.** Run a window manager (openbox) on the Xvfb display. Set Wine's Direct3D renderer to `gdi`. With the GL renderer, each DirectDraw blit was very slow.
- **Settings.** Edit `graalconfig.txt` (`gamemode=One Player Online`, `serveraddr=<loopback>`) rather than typing into the server combo box, which mangled the text.

Which builds start under Wine [live]

Each archive was unpacked and its client started offline for 15 s (`survey-all-builds.txt` in the project files).

- **Opens its window:** every build from the 1st to 15th Graal builds (including the 4th, 5th and 9th alphas), 1.1, 1.1.9, 1.28b2, 1.3 test, 1.31 to 1.39, 1.41.1, 2.0 (all revs), 2.01, 2.03, 2.04, 2.05 (all revs), 2.1 test 2 and 3, NewMain 24 and 26, NewWorld beta 1 and 2, and Graal 3D.
- **Exits at once:** 2.02, all three revisions.
- **NewMain 24:** opens a "File not found" box.
- **No client exe in the archive:** 1.21 and Zelda Online 2.

`client-internals/live-trace-205.md`

Live traces of the 2.05 rev 2 client

Date: 2026-10-29. These are behaviours seen in the running 2.05 rev 2 client: in Wine, offline with no network, with the player state read every 50 ms tick from outside the process. Hearts reports come from the 2.05 lab capture (`tools/lab-setup.md`, run 1) and from a loopback stub server. Tags: **[live]** seen in the running client, **[cap]** seen in a lab capture. This note has no addresses and no decompiler text. The harness, its scripts and the raw traces are in the project files under `graal-decompile/live-trace/`.

Hearts are reported by the client

- **On a hit, the client sends its own CURPOWER. [cap]**
    - In run 1, the player's own bomb hit them. In the same tick as the hit, the client sent one `PLI_PLAYERPROPS` message with CURPOWER = 5 half-hearts (down from 6), GANI `hurt` and SPRITE. Its knockback position updates followed.
    - Nothing from the server asked for it, and the server did not echo CURPOWER back.

    - The 2.22 lab shows the same thing (`lab-captures-2026-10-25/`): a hit sends CURPOWER + `hurt`; death sends the `dead` gani + STATUS; respawn sends CURPOWER by AP bracket.
- A client that applies damage locally but never sends CURPOWER leaves the server holding stale (full) hearts for that player.
- Not yet captured for 2.05: lava burns, heart pickups and respawn.
- Hearts are stored obfuscated in the client (`feature-specs/04-death-respawn.md`). A memory scan does not find them as a plain value. **[live]**

Ledge jump and running in place [live] (`new3.nw`)

- Walking up into the up-ledge at y = 14: the refused steps are counted 1 to 17, and the jump starts on the 18th.
- Jump y values from y0 = 14: 12.5, 11.3, 10.2, 9.5, 9.0, 8.8, 8.8, 9.0. This is exactly the curve in `ledges-push-pull.md`.
- **Running in place.** On every refused step the walk animation frame keeps advancing, and the gani stays `walk`. It changes only when the push gani takes over (at 18, against a wall) or the jump starts (at 18, at a ledge).
- While jumping the gani stays `walk`, and the blocked-step counter counts down 7 to 0 as the jump timer.

Water and lava [live] (`new4.nw`)

tile	what the client does
3644, dark-blue deep water	Blocks like a wall. `push` comes at the 18th refused step.
2882, bright-red lava	Swim: the gani becomes `lava`. Burns.
2958, dark-red lava floor	Walkable: the gani stays `walk` / `idle`. Burns.

- **Burn timing.** On either red tile a per-player counter runs 0 to 100 and wraps to 0 on the next tick. One cycle is therefore **101 ticks**, and each wrap costs half a heart. Three wraps took 3 → 1.5 hearts.
- Stepping onto grass resets the counter to 0.
- The liquid test uses the point (x + 1.5, y + 2).
- Burning to 0 hearts offline shows the `End` death scroll.

Server CURPOWER 0, death and respawn [live] (online, 2026-10-29)

The real 2.05 rev 2 client was logged in to a loopback stub server: login at (30, 30) on `onlinestartlocal.nw`, AP not sent. The player walked to (25, 42) before each death.

1. **The server sends CURPOWER 0.** The client at once sends GANI `dead`, and does not send CURPOWER. About 1.35 s later (27 ticks) it sends STATUS 0x1C, which sets the dead bit 0x08.
2. **A key after the timer.** The client sends `PLI_LEVELWARP` (0) to the same level at (30, 30). That is the login point, not the death spot. In the same frame it sends CURPOWER 6, GANI `idle`, X 30, Y 30 and STATUS 0x14 (dead bit cleared). It does not wait for a server reply.
3. **MAXPOWER 0 first.** If the server sends MAXPOWER 0 before the kill, the respawn still reports CURPOWER 6. The client does not cap its respawn hearts at a MAXPOWER of 0.
4. **CURPOWER 0 after each respawn.** The client dies and respawns the same way every cycle. A server that answers every revive with CURPOWER 0 (for example an account at MAXHP 0, which GServer-v2 clips to) puts a real client into a death loop.

`client-internals/live-trace-early.md`

Live runs of the pre-1.33 clients (1.1.9 to 1.32)

Date: 2026-10-30

These are captures of real clients and their bundled servers running under Wine, inside a network namespace that has loopback only. They check the static notes in `protocol/relay-1998.md`, `protocol/gserv-1999.md` and `client-internals/version-profiles/v121-v132.toml`. Tags: **[cap]** seen on the wire in these runs. The raw pcaps and decoded dumps are in the project files under `graal-decompile/live-trace/early/`.

How the runs were made

- Wine 9 (win32 prefix), Xvfb with openbox, Direct3D renderer set to `gdi`. Every pre-1.33 client opens its window under Wine.
- Servers: the 1.1.9 `Gserver.exe` (relay), GServer 1.28 (`GServerv.exe`, from the 1.28 beta 2 zip) and GServer13 (from the 1.3 test zip). Accounts were added in the server's Edit accounts window. The 15th Graal `.graal` levels were copied in, because the later zips don't ship `onlinestartlocal.graal`.
- Clients are pointed at the lab with `graalconfig.txt` (`gamemode=One Player Online`, `serveraddr` set to the loopback address). For 1.1.9 the connect dialog's default host `<list-host-classic>` was mapped to the loopback address.
- **1.21** ships only as an InstallShield 3 package. Its setup was run under Wine, and the installed folder (218 files) is kept in the project files under `graal-decompile/unpacked-installers/graal121/`. The installed `Graal.exe` and `GServer.exe` have the same MD5 as the copies analysed statically.
- **Keyboard under Wine.** These clients give DirectInput a keyboard data format that lists one object over a 256-byte buffer. Wine only fills listed objects, so arrow keys never reach the game, while chat and Alt keys still work because they arrive as window messages. A live patch that widens the format to 256 key objects in memory fixes this in 1.3 test and in 1.21. The exe checks its own checksum, so the file itself can't be edited. With the patch, Wine's dinput trace shows `dwNumObjs 256`. In 1.21 the player then walks. 1.28 beta 2 walks with the same patch, plus `joystickon=false` in `graalconfig.txt`. 1.1.9 walks with the same patch, and both 1.1.9 clients in a two-player run could move.
- **1.28 UDP receive under Wine.** At first only the 1.28 instance started first could move. The second stopped polling its keyboard right after login. The cause is in the game's UDP receive routine. It calls `recvfrom(sock, buf, 0x400, 0, &from, &fromlen)` without ever setting `fromlen`, a local variable that held a stack address (about 3.2 million) in the traced runs. When the first datagram arrives (an arrow from the other client), Wine fills the address buffer out to that length and writes past the top of the main thread's stack. The exception chain on the stack is then overwritten, and the thread loops through nested access violations until the stack overflows. The window stays open and Wine keeps passing it key events, but the game loop is dead. The first instance never got a datagram (the second died before sending one), so it kept running. Windows checks only that `fromlen` is at least 16 and writes 16 bytes, so the bug would not show there (*inferred*). An in-memory patch that sets `fromlen` to 16 before the call fixes it. With the patch on both instances, both players walk and UDP flows both ways. The patch script is `udpfix.py` in the project files.

GServer 1.21 with 1.21 (GSERV007)

[cap]

- The login frame has the same shape, with `GSERV007`. The installed server ships `accounts.txt` with two accounts, `guest`/`guest` and `super`/`super`. Each line holds `chr(len+32)` + name, `chr(len+32)` + password, then the starting props in the same encoding as id 9.
- The reply is **one zlib frame** holding 9 (own props), 14 (warp to `level1.graal`), 100, 101 with the board, 6 (level name), 1 (links) and 10. 1.28 and 1.3 test put 101 in a frame of its own. There is no 39, as in 1.28.
- Walking sends one id 2 message every 100 ms with X/Y and SPRITE, as in 1.3 test.
- The installer pops up "No MCI device open." under Wine (no MIDI device). Pressing OK continues.

GServer13 with 1.3 test (GSERV014)

Run by the live-trace lab. **[cap]**

- The first client frame is a u16 big-endian length followed by a zlib stream holding `" GSERV014" + chr(5+32) "guest" + chr(5+32) "guest" "\n"`, as in `gserv-1999.md`.
- A bad account gets id 16 with "An account with the name and password you typed in doesn't exist."
- Reply order after a good login: 9 (own props), [28 (flags), only when the account has flags], 14 (level warp: x, y, level), 100 (raw size) and then a separate zlib frame holding 101 and the board, 6 (level name), 1 (links), 5 (signs), **39 (level modification time)**, 3 (NPC props, scripts included), 10.
- The client then asks for NPC images with 23, sets flags with 18, and adds the level's horses with 7. The server answers 30 (file send failed) for the missing images, and deletes the horses with 18 when the horse lifetime runs out (10 s).
- Walking sends one id 2 (props) message per 100 ms with X and SPRITE. Chat is id 2 with prop 12.

GServer 1.28 with 1.28 beta 2 (GSERV013)

[cap]

- The login frame has the same shape, with `GSERV013`.
- The reply order is the same as 1.3 test, except that **there is no id 39**. The level-timestamp cache is new in 1.3 (the 1.3 notes: "levels will be saved on the local hard disk … they contain a timestamp").
- The own-props string (id 9) is one prop shorter than in 1.3 test, which ends with an extra prop (`@R`). That fits a parser that stops at prop 31 in 1.28 beta 2 and 32 in 1.3 test (`v121-v132.toml`).
- A second login with an account that is already online gets id 16 "The account is already in use." The account's first session keeps playing.
- **Second player.** The newcomer gets its own 9 and then, in the same frame, an 8 (other player's props) for the player already in the level, before the 14 warp. The player already there gets 8 messages about the newcomer, starting with the nickname and level. The ids in those 8 messages are the server's player numbers (1 and 2 here). Where the UDP address and port are carried has not been decoded. The other-player props are the obvious place (*inferred*).
- **UDP.** Once both clients are in the same level they send UDP datagrams straight to each other's port, without going through the server. Each datagram is one uncompressed message, `chr(id+32) + data`.
    - **Walking:** the moving client sends **UDP id 0** to the other player every 100 ms. The five bytes are the player props X, Y, SPRITE, STATUS and CARRYSPRITE (15 to 19), each +32. That is the same field list as UDP id 0 in 2.05 (`protocol/udp-p2p.md`).
        - X and Y are in half tiles; Y went up by 2 per step while walking down.

        - SPRITE is animation frame·4 + direction (0 up, 1 left, 2 down, 3 right). Walking down cycled 22, 14, 6, 14 and stopped at 2.
        - STATUS was 16 (`0x10`) and turned 20 (`0x14`) while the player crossed the lake south of the start, then back to 16 past it. At the same moment the client sent prop 18 = 20 over TCP. So `0x04` is the swimming state in this build (*inferred* from where it flipped; later servers name `0x04` "male").
        - The trailing `0x1F` is CARRYSPRITE −1 (nothing carried).
    - At the same time the walking client sends TCP id 2 to the server with only the changed props (16, 17 and, when it flips, 18). **The server forwards nothing to the other player.** The other client sees the walk only through UDP. That is the "player positions & sprite numbers … sent directly to all players that are in the same room" of the 1.28 notes.
    - **Arrows:** id 12 twice every 0.5 s is **arrow add**. The level `onlinestartlocal.graal` has two NPCs whose scripts run `shootarrowleft` and `shootarrowup` on a 0.5 s timeout. The payloads are x, y (half tiles, at each NPC), direction (1 left, 0 up), a type byte, and a last byte sent as `v+33`, the same layout as the later `ARROWADD` (`protocol/packet-layouts.md`). Only one client sends them, to every other player in the level. In a three-client run the first player in the level sent them to the other two. When it quit, the player who had joined second took over within a second and sent them to the third. So 1.28 has one player per level that runs the NPC timeouts for everyone, and the role passes on when that player leaves. **The server picks that player** with id 10 (`ISLEADER`, no payload). The first player's login reply ended with 10, and the second and third players' replies had none. When the first player quit, the server sent 10 to the player who had joined second, and then sent 8 with prop 51 (player gone) about player 1 to both remaining players. The arrows from the new leader started straight after. 1.21 and 1.3 test also end the first player's login reply with 10, so the leader role is older than UDP.
    - A newcomer sends one UDP id 22 with no payload to each player already in the level (seen in the two- and three-client runs). 2.05 uses 22 for chat, so this is most likely the newcomer's empty chat text (*inferred*).

GServer13 with 1.31 (GSERV016) and 1.32 (GSERV018)

[cap]

- Both clients send the same login frame with their own token.
- GServer13 refuses both with id 16 "You need at least Graal version 1.3 to play on this server." A token newer than the server's list is treated as too old. It does not cause a disconnect without a message, or an "unknown version" error.
- **Saved password.** 1.31 and 1.32 (and, by string search, 1.3 test to 1.39) save the login under `HKCU\Software\Graal\Client`: `accountname`, `nickname` and `password`. The password is the 48-character DES string form from `saved-passwords.md`, with the same key that 2.05 uses. `tools/file-formats/graaldes.py` turned the value from this run back into `guest`. So the warning in `saved-passwords.md` applies from 1.3 test on, not only to 2.x. 1.28 beta 2 still reads `accountname=` and `password=` as plain text from `graalconfig.txt`.

1.1.9 Gserver with two 1.1.9 clients (relay)

Server started with "Use local levels". **[cap]**

- **Each client opens two TCP connections.** One joins the game `Graal8` and stays in it. The other first joins `G8` with no level, then leaves (`-`, `C`) and re-joins as `G8onlinestart.graal`, then `G8onlinestartlocal.graal`, as the client moves through levels. All lines go to the same server port.

- After every `N` the server answered `NR`, then `NL`, then `I<id8>`. Ids are `0000000n`, numbered across all connections in the order they connect (the three-client run below gave 1 to 6). A connection keeps its id when it changes room.
- **Both connections pair up.** When the second client joined, each connection got `+<id>def` for the other client's connection in the same game. The client answered `P<id>`, and the server confirmed with `P` to both sides. That happened first on `Graal8` and then on `G8onlinestartlocal.graal` once both clients were on that level. The `Graal8` pair carried only the first state push (6 `D` lines each way). The level pair carried the rest (about 120 `D` lines each way in a minute). So the per-level room is where play traffic goes, as `relay-1998.md` says, and the fixed `Graal8` room is an extra channel. What the client uses that channel for beyond the first push was not seen in this short run.
- The state push after `P` is, in order: `L` (level, sent twice as the client moves on), `P` (position), `N` (nickname), `S` (colours), `7` (pose flag), and later `T` (10-character login time), `Y` (sword/shield power) and `C`.
- `D` lines carry the sender id as two base-128 characters (`"  "` = 0, `"!"` = 1 …), as in `relay-1998.md`.
- **Type `A` is an arrow shot.** Each client sent `A` about four times a second, alternating two 5-byte payloads. They are the arrows of the two shooting NPCs in `onlinestartlocal.graal` (see the 1.28 UDP id 12 notes above). Unlike 1.28, both 1.1.9 clients sent them, because each client runs the level's NPC scripts itself.
- **Walking.** With the keyboard patch both 1.1.9 clients walk. The moving client sends `D` type `P` on the level connection every 100 ms: x and y in half tiles, then frame·4 + direction, the same encoding as SPRITE in 1.28. Walking right gave `^\3`, `` `\3 ``, `b\+` … (x up by 2 per step, y fixed); stopping sent the idle frame (`#` = 3, facing right).
- **"Use levels from cyberjoueur".** With that option the server sent `NR` alone after each `N`, with no `NL`. The client then joined `G8onlinestart.graal` and stayed there, instead of moving on to `onlinestartlocal.graal`. It opened no other connection (no HTTP).

1.1.9 Gserver with three 1.1.9 clients (relay)

Server started with "Use local levels". **[cap]**

- Every connection pairs with every other connection in the same room. When the third client joined `Graal8`, the server sent it `+` for both existing `Graal8` connections, and sent each of them `+` for the newcomer. The same happened in the level room. Each pair then exchanged `P`. So a relay game is a full mesh of pairs, and every `D` line goes to all the sender's partners.
- Each time the newcomer's level connection joined a room, the server first sent it `C<id>` for every other connection id, including ones it was about to pair with, then the `+` lines. This looks like a reset of old partnerships before the new ones (*inferred*).
- When a client quit, the server sent `C<id>` for its level connection to the partners in that room, and `-<id>` for both of its connections to every connection, in all rooms. The other clients carried on, and the remaining ones could still walk.
- **No leader.** All three clients sent the NPC arrows (`A`) all the time, and the two remaining clients went on doing so after the first quit. In the relay each client runs the level's NPC scripts for itself. The leader in 1.21 and later comes from the GSERV server (id 10, above).

What this changes

- `gserv-1999.md`: the login reply order is now the captured one, plus the 1.28 difference (no 39), the refusal of newer tokens, the second-player 8 messages and the UDP shape.
- `relay-1998.md`: the two connections, `NR` then `NL` (or `NR` alone with Cyberjoueur levels), `A` = arrow shot, and the `P` sprite encoding.

- `gserv-1999.md`: UDP id 0 = X, Y, SPRITE, STATUS, CARRYSPRITE, id 12 = arrow add, and id 10 = level leader.
- `v121-v132.toml`: new `level_leader` feature.
- `v121-v132.toml`: `credentials_in_registry` is now confirmed and names the DES string form.

Open

Nothing from these runs is left open.

`client-internals/newworld-betas.md`

NewWorld beta clients (graalnwb1, graalnwb2)

Date: 2026-10-21

The two NewWorld betas are forks of mainline client builds:

Build	Linked	Code base	Compared against
graalnwb1 (`Graal-NW.exe`, MD5 88d1f84c…)	2001-08-06	2.05	2.05 rev2
graalnwb2 (`Graal-NW.exe`, MD5 af2d2106…)	2001-10-03	2.1 test	2.1 test 3

Both betas went through the same Ghidra pipeline as `build-diff-204-205.md` (Delphi compiler spec, RTTI names, switch-table recovery, feature export, `tools/build-diff/fdiff.py`). Each beta was compared with its own base, so what's left over is the NewWorld delta. Every change below appears in **both** betas unless the text says otherwise. This note is cleaned: it has no addresses and no decompiler output.

Pair	Functions (old / new)	Identical	Changed	Only old	Only new
2.05 rev2 → nwb1	6647 / 6494	5917	423	307	154
2.1 test 3 → nwb2	6703 / 6579	6051	354	298	174
nwb1 → nwb2	6494 / 6579	5715	611	168	253

The nwb1 → nwb2 row is mostly the 2.05 → 2.1 base change. The NewWorld-specific code is the same in both betas.

Classes

Added (both betas): `TLevelHeights`, `TAdventureBacktileArea`, `TDrawText`. Each is a plain `TObject` with no published methods.

Removed (both betas): `TComputer`, `TEditBaddy` (baddies), `TAdventureHorse` (horses), `TEditorChest`, `TChestEdit` (chests), `TAdventureCarry`, `TAdventureBall`, `TPos`, `TWebClient`. This matches `readmenw.txt`: "horses are npcs now", "there are no built-in horses, carry object, freezeplayer etc. anymore, see the control-npc script for it". Baddies and chests are scripted too.

All other class differences between the two betas (`TEditNPC`, `TEditVariable`, `TEditorDefNPC`, `TSmilie` and others) come from the 2.05 → 2.1 base change and are not NewWorld code.

Instance sizes:

Class	2.05 rev2	nwb1	2.1 test 3	nwb2
`TAdventureLevel`	184	**33968**	184	**33968**
`TGraalMap`	76	96	256 (now a `TAdventureLevel` subclass)	34060
`TServerPlayer`	336	320	364	348
`TPlayer`	800	760	836	792

The 33968-byte level: a 65 × 65 height grid

`TAdventureLevel` in the betas is a 168-byte header followed by a **65 × 65 grid of doubles** (65 × 65 × 8 = 33800; 168 + 33800 = 33968). The grid holds one height per tile corner of the 64 × 64 board, row-major with a stride of 65.

The 168-byte header has five fewer object lists than 2.05's (most likely the lists for the removed baddy, chest, horse and carry objects) and one new list, the backtile areas (see below). The other fields move to fill the gap. The board is still a separate `TBoard` (4096 tiles), as in 2.05.

In nwb2 the 2.1 base had already made `TGraalMap` a subclass of `TAdventureLevel`, so the gmap inherits the 33968-byte level part and adds its own 92 bytes.

Where the heights come from

Heights apply only when the current level is part of a gmap that has a height file, and only in one game mode (internal value 5; which mode that is wasn't confirmed). Otherwise every height reads as 0.

1. **Map corners.** When a gmap loads, the client looks for a sibling file named `<gmap base>_heights<ext>`, so `world.gmap` gives `world_heights.gmap`. (The shipped samples are named `..._heights.txt` next to a `.txt` map, for example `levels/test3d/test3dmap_heights.txt`.) The file is a CSV grid of (width + 1) × (height + 1) quoted entries, one per level corner:

   ```
   "100:36646","50:29374","100:97473","100"
   "50:4646","-30:342","-50:12393","-50"
   ```

   Each entry is `height` or `height:seed`. The seed in the entry at (x, y) belongs to the level whose top-left corner that is. The entry at the bottom-right corner (width, height) holds the map-wide seed.

2. **Level edges.** Each edge between two corners is filled by 1-D midpoint displacement out to 64 steps. The midpoint gets the average of the two ends plus `(random − 0.5) × 2 × amplitude`. The amplitude starts at 4.0 and is multiplied by 0.6 at each level of subdivision. The RTL random seed is set to the map-wide seed first. Neighbouring levels share their edges, so seams match.

3. **Level interior.** When a level is entered, its four 65-sample edges are copied from the map. The RTL random seed is set to that level's seed, and the interior is filled by recursive subdivision. Each step displaces the centre column and centre row, then recurses into the four quadrants. The amplitude starts at 4.0 × 0.6 and also shrinks by 0.6 per step. The terrain is deterministic: the same file always gives the same heights.

4. **Per-level control grid (`TLevelHeights`).** The gmap holds one `TLevelHeights` per level: an enabled flag and a 9 × 9 grid of doubles (one point every 8 tiles; 4 + 4 + 81 × 8 = 656 bytes). When the flag is set, each 8 × 8 cell gets a bilinear correction, so the generated surface passes exactly through the 81 control points.
    - nwb1 creates these objects but nothing in it sets the flag, so the control grid is inert there.
    - nwb2 fills them from a `HEIGHTS` … `HEIGHTSEND` block in the gmap parser (the generator form in `formats/grmap001.txt`). That block sets the flag.

Using and changing the heights

- Drawing players and other level objects looks up the heights of the four corners of the tile under the object.
- **`setz x,y,w,h,z1,z2,z3,z4`** writes heights into the grid itself. Every grid point in [x, x+w] × [y, y+h] gets a bilinear blend of the four corner values. A zero width or height averages the corners instead. Points outside 0..64 are skipped.
- **`updateterrain`** re-clamps the view and redraws the terrain surface. The `readmenw.txt` workflow is "change `waterheight` / `groundheights[]`, then call `updateterrain`". Those two variables are in nwb2 and the 2.1 base, not in nwb1.

In 2.05 itself `setz`, `updateterrain` and `setbacktile2` are parsed but have no executor case (see `build-diff-204-205.md`). The betas are the builds where they do something.

Backtile areas (`TAdventureBacktileArea`)

`setbacktile tile` sets one level-wide "back tile", the tile drawn behind trees and other foreground tiles. **`setbacktile2 tile,x,y,w,h`** adds a 24-byte area record (tile, x, y, width, height) to the level's area list and redraws the board. To find the back tile for (x, y), the client scans the areas in the order they were added and uses the first one whose rectangle contains the point. If none does, it uses the level-wide back tile. Both commands work on the current level in play and on the editor's level.

Deferred text (`TDrawText`)

While a frame is drawn, the client keeps a temporary list. Any nickname or chat text drawn during the scene pass is not drawn right away. It is queued as a 28-byte `TDrawText` holding the text, x and y as doubles (offset by the level's position in the gmap), and the colour. After the scene (terrain, objects, overlays) is finished, the queue is drawn in one final pass and freed. Outside a frame, text is drawn immediately as in 2.05. The likely purpose is to keep raised terrain and neighbouring levels from covering names. It matches the beta changelog line "Nicknames/ Chat texts are displayed in the correct level."

GS1 changes

The command table keeps the 2.05 layout: 12-byte records (name, a flags word, an argument-signature string). It has **161 commands in 2.05 and nwb1** (`say` … `hide`) and **165 in nwb2**. After it come a 4-name direction list (`up`, `left`, `down`, `right`), then, in both betas only, an **8-name NW time-variable list**:

Variable	Range (from `readmenw.txt`)
`nwday`	1…28
`nwweek`	1…40
`nwyear`	1000…
`nwweekday`	1…7
`nwmonth`	1…10

Variable	Range (from `readmenw.txt`)
`nwtime`	0…1439 (minutes)
`nwhour`	0…23
`nwmin`	0…59

These are script variables (also usable as `#v(name)` message codes), **not commands**. The calendar has 28-day months, 10 months and 40 weeks. The server sends the time (packet 42), and "2000/5/1 realtime was 1000/1/1 graal time".

String-list commands (nwb2 only): `addstring`, `insertstring`, `removestring` and `deletestring` are inserted after `loadmap`, with the signatures shown here:

Command	Arguments
`addstring list,text`	string, string
`insertstring list,index,text`	string, number, string
`removestring list,text`	string, string
`deletestring list,index`	string, number

They go with `sarraylen(list)`, `lindexof(list,text)` and `#I(list,i)`, which are also only in nwb2. Lists are stored as normal string flags in comma form (`numbers="12 34",567,"8,9"`). nwb1 has none of this. Its `readmenw.txt` doesn't mention string arrays either; the section appears only in nwb2's copy.

Executor cases. These counts compare the betas with their bases. They are the same for both pairs, apart from nwb2's four string commands.

- Live in the betas, ignored in the base: `setfocus`, `resetfocus`, `setbacktile`, `drawovertrees`, `setbacktile2`, `setz`, `updateterrain`.
- Parsed but ignored in the betas, live in the base: `removecompus`, `putcomp`, `putnewcomp`, `hitcompu` (baddies); `puthorse`, `takehorse`, `takeplayerhorse`, `removehorse` (horses); `carryobject`, `throwcarry`, `takeplayercarry`; `setsword`, `hidesword`, `setbow`; `freezeplayer`; `enabledefmovement`, `disabledefmovement`; `toinventory`; `followplayer`. The control NPC script (`controlnpc.txt`) takes over movement, swords, carrying and horses.

Packets

Server → client (TCP)

The receive switch covers 0…168 in 2.05 and 0…169 in 2.1. Compared with its base, each beta **drops six receivers**:

Id	2.05 meaning	Why it goes
2	BADDYPROPS	no built-in baddies
4	LEVELCHEST	no built-in chests
17	HORSEADD	horses are NPCs
18	HORSEDEL	horses are NPCs
154	FREEZE	no built-in freeze; `freezeplayer` is gone too
155	UNFREEZE	as above

Everything else in the base's receive set is still there. nwb2 also has ids 49 and 169, but they come from the 2.1 base.

Client → server (TCP)

Several 2.05 senders have no matching function in nwb1: HORSEADD (7), BADDYPROPS (15), BADDYHURT (16) and OPENCHEST (20). The bomb, arrow, item-add and item-take senders match 2.05 exactly. "No match" is an upper bound (see the matching caveat in `build-diff-204-205.md`), but it agrees with the removed classes.

Peer-to-peer UDP

The UDP receive switch covers 0…26 in 2.05 and 0…27 in 2.1. Both betas drop **16 and 17**, the horse add/remove pair that mirrors TCP 7/8 (see `protocol/udp-p2p.md`). They keep 0 and 10–15 and 18 up to the end of the range. nwb2 keeps the 2.1 addition, 27.

List server

Both betas hide the list-server host with the same DES string as 2.05 (`<list-host-classic>`; see `des-and-hidden-strings.md`). They build the port and list name one character at a time, as 2.05 does:

Build	Port	List name sent with id 0
2.05 rev2	14921	`newmain`
nwb1	14921	`newworld`
2.1 test 2/3	14922	`newmain`
nwb2	14922	`newworld`

So the port comes from the code base (14921 in the 2.05 line, 14922 from the 2.1 tests on), and the list name is the only NewWorld change. The betas log in to game servers with the token `G2N15041` instead of the mainline `GNW03081` (see `build-catalogue.md`).

Live lab runs (2026-10-30)

Both betas were run under Wine (32-bit prefix, virtual X display) in a network namespace whose hosts file points the GraalOnline names at a local stub. The stub answers as list server (14921 to 14923) and as game server, and logs every packet. The 2.1 test 3 build was run the same way for comparison. The per-version results are in `version-profiles/141-217.toml` (`vnwb1`, `vnwb2`).

List server

Build	Port	What it sends
nwb1	14921	id 0 `newworld` only
nwb2	14922	id 0 `newworld`, then id 1 with account and password (two GSTRINGs)
2.1 test 3	14922	id 0 `newmain`, then id 1 with account and password

So sending the account to the list server begins with the 2.1 tests (September 2001), not 2.17. nwb1, on the 2.05 code, does not do it. nwb2's start window is GraalOnline-themed with Account and Password fields. In one run it sent 23 characters of a 28-character password; the cut-off was not pinned down.

Game server login

Both betas send client type 0, the key byte, `G2N15041`, the account and the password, then LANGUAGE (37) `English`. Their first props are the same six as 2.05: NICK (0), HEADGIF (11), COLORS (13), STATUS (18), UDPPORT (31) and ADDITFLAGS (33).

- **COLORS has seven bytes**, the same in every run, where 2.05 has five. Both betas' prop parsers read eight colour bytes. A server must take the length from the prop and must not echo seven bytes to a client that reads a different count.
- HEADGIF comes without an extension (`headalgo000`).
- STATUS has a new bit, 0x20.
- The first login sometimes stalled with the level drawn but no HUD or player and no props sent. Logging in again fixed it every time.

After joining

- **Files.** nwb1 asks WANTFILE `shield1.png` and UPDATEFILE `bodynw.png` and `timenums.png`. nwb2 also asks for `idle.gani` and `headalgo000`. Neither asks for the sword, bomb and bow images 2.05 asks for.
- **Movement.** GANI (10) goes out as a string. While idle the client re-sends `idle` about every 100 ms, because the control NPC calls `setani` each tick (2.05 sends it only now and then). Walking sends X and Y in half tiles plus SPRITE, and nwb2 re-sends `walk` on each step.
- **Keys.** S does nothing online, so a server has to give the sword as an NPC weapon. A plays the grab animation. The control NPC comes from the local `controlnpc.txt` as the weapon `Control-NPC`, added when the player is created.
- **Time (packet 42).** The payload is a GINT4: minutes since 1000/01/01 00:00 on the NW calendar (28-day months, 10 months, 280-day year). A value of 162310450 showed as 1402/06/16 14:10 on the HUD clock.
- **Offline Training** in nwb1 runs without a server (HUD 150/150, clock 1000/01/01).

Client → server set

A scan of the send sites (same method as `version-profiles/README.md`) shows both betas drop seven ids that their bases send: 7, 8, 11, 15, 16, 17 and 20 (horse add/delete, throw carried, baddy props/hurt/add, open chest). That widens the four listed under Packets above. The prop parsers and writer otherwise match each base, and there is no baddy-props parser.

Corrections to other notes

- `build-catalogue.md` credited "string arrays" to nwb1. They are nwb2-only. nwb1's distinctive features are the terrain heights, backtile areas, deferred text and NW time variables.
- The 2.05 GS1 table has 161 commands. A count of 163 reads two entries past the end (`up` and `right` from the direction list that follows).

`client-internals/offline-play.md`

Offline play notes

Date: 2026-10-14

Requirement

Seihai must support playing levels offline (no listserver / gserver connection).

Observations from original client

- Tutorial levels are embedded in the executable (not loaded from disk).
- `2dadventureclient` already demonstrates local player + level simulation using the shared game object model.
- Level files, GANIs, NPCs, and links can be loaded from a local `levels/` tree.
- The original client's editor (F4) and snapshot features operate on local level data.

Current status

- `.nw` parser + board-modify applicator (`Level::apply_board_modify`).
- `OfflineSession` can load a level and apply `BoardModify` updates.
- GANI parser extracts SPRITE defs, ANI frames (placements), and DEFAULT* fields.

Remaining for offline play

1. Tile collision using the parsed boards.
2. NPC script execution (seihaiscript).
3. Rendering the tile data.

See `offline-tutorial-205.md` and `level-editor-205.md` for what the 2.05 client itself does offline.

`client-internals/offline-training.md`

Offline training levels

Date: 2026-10-14

Derived from observed client behavior. No decompile material.

Observation

Tutorial levels are embedded in the executable. They can be drawn and walked without a network connection. The client does not require a listserver or gserver to enter them.

Difference from session levels

The global that holds the current level name is set only by the session "set level" message (the byte that arrives after a successful world login). Offline training levels never set that global. A debugger session that shows a training level drawn while the level pointer remains null is expected and correct.

Enter path

Entering a training level stores the level name on the player object and the arrival coordinates. Nearby links (for example to a house interior) are present in the embedded data. The same collision and script paths used by disk levels are used here.

Offline implication

An offline session that loads a GLEVNW01 file from disk is already past the embedded-tutorial case. The same collision, sign, and script paths apply. The only difference is the source of the level data (disk versus embedded).

The offline slice therefore does not need to special-case training levels. It only needs to:

1. Load a level from disk (already done).
2. Apply the tile walk rules (see `formats/tile-walk.md`).
3. Execute the common GS1 commands on touch (see `npc-library/gs1-commands.md`).
4. Blit the selected sheet (see `rendering-observations.md`).

Once those four are in place, both embedded training levels and disk levels will behave the same way.

See `offline-tutorial-205.md` for the 2.05 details: the *Play offline* path, the resource lookup, and the level-by-level contents.

`client-internals/offline-tutorial-205.md`

Offline mode and the embedded tutorial (client 2.05 rev 2)

Date: 2026-10-21

Derived from the 2.05 rev 2 binary (resources, strings, and the offline and level-load paths) and from decoding the embedded levels with `formats/grv1-decoder/grv1.py`. Names only, no code. Tags: **[2.05]** · **[2.22]** · **[inf]**.

This note refines `offline-play.md` and `offline-training.md`.

How the client goes offline

Offline play is reached in these ways:

1. **Training on the start screen.** It starts the embedded tutorial at `tutorial_b-4.graal` with no connection (confirmed live, see `live-checks-editor-offline-sound.md`). **[2.05]**
2. **"&Play offline" on the reconnect dialog** (`TReconnectFrame`). The dialog appears when the connection fails or drops ("The connection could not be established!", "You have been disconnected from the game server.", "There are too many players online."). Choosing *Play offline* frees the game socket, so the connection object is nil from then on. **[2.05]**
3. **Command line.** `Graal.exe level.graal` starts with a local player and opens that file in the editor. Play then runs it offline (see `level-editor-205.md`).
4. **F5 Load level** while online. The client warns "When you load a level you will be disconnected from the server. Continue?" and then loads the file locally.

"Offline" in the client simply means *no game connection object*. Every check that refuses the editor, enables the tutorial resources, or sends file requests tests that one condition.

Where the player lands

When *Play offline* is chosen:

- If the level the player was on exists in the local folder, the player stays on it at the same coordinates.

- Otherwise the player is moved to **`tutorial_b-4.graal` at (32, 60)**. The map overlays are switched to the tutorial set: `tutorialmap.png` / `tutorialmap.txt` for the big map and `tutorialmap2.png` / `tutorialmap.txt` for the minimap. A "first offline start" flag is also set.
- If the level still fails to load, a warning names the level and the client stays at the reconnect dialog.

Both of the first two cases were seen live: after a refused connection the player went to the tutorial, and after a drop on a level that was also on disk the client reloaded that level at the same spot (see `live-checks-editor-offline-sound.md`).

Where offline levels come from

The level loader checks two sources in order. **[2.05]**

1. **Embedded tutorial.** When offline, and the name starts with `tutorial`, the loader maps the file name to an `RCDATA` resource and reads the level from the executable.
2. **Disk.** Otherwise it looks the name up through the file-name cache and reads the file from disk.

The embedded copy wins: with a `tutorial_b-4.graal` of different content in the game folder, the client still loads the embedded level and does not look for the file (live).

Online, names starting with `tutorial` are not special. They go to the server like any other level.

Level file	Resource
`tutorial_a-1.graal` … `tutorial_a-4.graal`	`TUTORIALA1` … `TUTORIALA4`
`tutorial_b-1.graal` … `tutorial_b-4.graal`	`TUTORIALB1` … `TUTORIALB4`
`tutorial_b-2_house.graal`	`TUTORIALB2H`
`tutorial_c-1.graal` … `tutorial_c-4.graal`	`TUTORIALC1` … `TUTORIALC4`

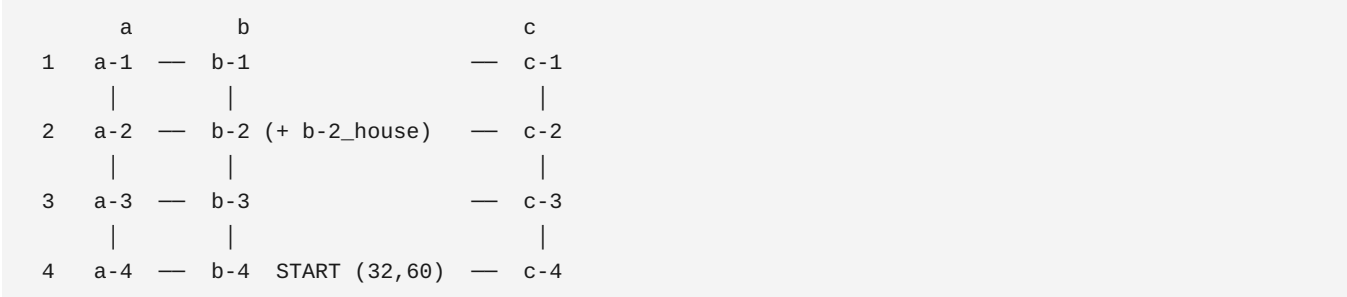
All 13 are `GR-V1.03` binary levels (see `formats/level-binary.md`).

Special level names

Three level names trigger an extra setup step when loaded: `level1.graal`, `YoshiClanR2.zelda`, and `YoshiClanFields.zelda`. These are the original single-player start levels. What the step does has not been traced. **[2.05]**

The tutorial world

The levels form a 3×4 grid: letters are columns from west to east, digits are rows from north to south. Edge links use `playerx` / `playery` to keep the other coordinate, landing at 0 or 61 on the far side.



Level	Links	NPCs	Chests	Signs	Content
a-1 … a-4, c-1 … c-4, b-1	2–3	0	0	0	Scenery only
b-4 (start)	3	8	1	3	Welcome sign ("In this tutorial you will learn … say the password 'Balibu'"). A gate NPC opens when the player chats `Balibu` and stands in a y range. A `setmap tutorialmap.png …` / `setminimap …` NPC. Two light NPCs (`drawaslight`, `setcoloreffect`). Two character NPCs. The chest holds `glove1` (the red power glove) with sign 0: "Go to the stone, press … to lift it!"
b-3	4	0	0	1	Sign at (36, 37): "Use the sword to cut those trees!"
b-2	5	5	0	2	An arrow box NPC (`lay darts`). A door NPC that opens on the `opendoors` flag and plays `chest.wav`. Character NPCs. The link into the house is at (22, 26). The signs explain shooting statues to open the door, then "Excellent! You got the treasures! … Press F8!"
b-2_house	1	6	6	7	A shopkeeper ("give me 5 rupees", how to drop a rupee). A house gate NPC that slides open on the `housegateopen` flag with `stonemove.wav`. Two lamps, two lights. Six chests: battleaxe, lizardshield, spinattack, nukeshot, fullheart, superbomb, each with its own sign.

The tutorial ends by sending the player to **F8** (server list) to go online.

Signs at (0, 0) are not drawn on the board. Chests and the `say n` command show them by index. Six of the seven house signs are chest signs.

Later clients

- **2.17, 2.22 and 2.31** keep embedded tutorials but trim them to a 3×3 grid of 9 levels named `offlinetutorial_a-1.graal` … `offlinetutorial_c-3.graal` (resources `TUTORIALA1` … `TUTORIALC3`, still `GR-V1.03`). Training starts on `offlinetutorial_b-2.graal`, the centre of the grid (2.22 and 2.31, live). **[2.17][2.22][2.31]**
- Offline, every client tested (2.05, 2.22, 2.31) reads `levels/tiledefs/tiledefsOffline.txt`, because the tile-definition file name is `tiledefs` + server name + `.txt` and the offline server name is `Offline`. The shipped file only targets `offlinetutorial*` levels, so it has no effect on the 2.05 tutorial. **[live]**
- The standalone GraalEditor 2.3.1.1 carries the same 9 `offlinetutorial_*` names and `tiledefsOffline.txt`. **[inf from strings]**

Offline implications

- An offline reimplementation that wants the original first-run experience needs the 13 resources and the `tutorial*` resource lookup. Decoded copies can be produced locally with the decoder. The levels are not committed here.
- The resource mapping applies only when the client has no connection, so a server can host its own `tutorial_*.graal` levels without conflict.

`client-internals/player-mannerisms-live.md`

Player mannerisms seen in real clients (1.39, 1.41, 2.05 rev 2, 2.22)

Date: 2026-10-30. What the player does, tick by tick, in four real clients run under Wine with no network.

- **2.05 rev 2.** The player state was read every 3 ms from outside the process while xdotool held keys, and the screen was recorded at 60 fps. Tag **[live]**. The test level is `lab1.nw` (described below).
- **1.39, NewMain 1.41 (`graalnm24`) and 2.22.** These were traced the same way, from their own player records (see "Other builds" below). Tags **[1.39]**, **[1.41]**, **[2.22]**. Where only video was used, the tag is **[video]**, and those counts are good to ±1 tick.
- **The gani files.** These are the files that the timings come from. Tag **[gani]**.

A tick is the 50 ms game tick. Speeds are in tiles. This note has no addresses and no decompiler text. The harness, the level, the raw traces and the gani comparison script are in the project files under `graal-decompile/live-trace/mannerisms/`. Stills are in `mannerisms-shots/`.

Summary table (2.05 rev 2)

action	what the client does	timing
walk	`walk`, 0.5 tile per tick, one gani frame per tick (8 frames)	8-tick cycle
key released	`idle` in the same tick, frame 0	0 ticks
run into a wall	`walk` keeps cycling in place; `push` on the **18th** blocked tick	0.9 s
push pose	5 frames × 4 ticks; restarts at frame 0 each time the 20-step push cycle wraps	20-tick loop
slide along a wall	holding Down into a wall and adding Right walks right at full speed with `walk`. The blocked count drops to 1 and stays there.	—
lift (A)	`lift` for **6 ticks**, then `carrystill`. The object counts as carried from the 5th tick.	0.3 s
carry walk	`carry`, still 0.5 tile per tick, 8 frames × 1 tick	—
hide (D while carrying)	`hiddenstill` / `hidden`; walking is **0.1 tile per tick**; only the object is drawn	—
throw (A, or S from hiding)	`idle` in the same tick	0 ticks
sword (S)	`sword` 5 frames × 1 tick, then `idle`. The player stops while it plays.	5 ticks
spin (S again during a swing, has-spin status)	the swing is cut off in the next tick and `spin` plays: 13 frames × 1 tick, then `idle`	13 ticks (0.65 s)
hurt	`hurt` for 31 ticks (12 if an arrow is held); knockback 1 tile per tick for 5 ticks; blinking and no further hits for 40 ticks	1.55 s / 2.0 s

action	what the client does	timing
swim	`swim` from the first tick the point (x + 1.5, y + 2) is on water; 0.5 tile per tick; 8 frames × 2 ticks; **keeps animating when standing still**	16-tick cycle
grab / pull	`grab` on A at a wall. `pull` starts on the arrow away from the wall: 3 frames × 2 ticks, with a pull try every 18 ticks.	—

Walking, idle and running in place [live]

- Each walk tick moves 0.5 tiles and advances the `walk` frame by one (0 to 7, then 0). The first tick after the key goes down already moves the player.
- Letting go of the arrow switches to `idle` frame 0 in the next tick. There is no slow-down and no finishing step. In `idle` nothing in the player state changes from tick to tick.
- **Against a wall.** A refused step still advances the walk frame, so the player runs in place. The blocked-step count goes 1, 2, …, 17, and `push` replaces `walk` on the 18th. This is the same count as the ledge jump (`ledges-push-pull.md`). Deep water (tile 3644) counts as a wall.
- **The push pose.** Each push frame lasts 4 ticks (`push.gani`: 5 frames, each `WAIT 3`). When the count reaches 38 it goes back to 18, and the gani starts again from frame 0. Five frames of 4 ticks make exactly that 20-tick cycle, so the pose loops seamlessly.
- **Sliding along a wall.** Still holding Down into the wall, add Right. The player walks right at the full 0.5 tile per tick in `walk`, and the push pose is gone in that tick. Let go of Right and the player runs in place again from walk frame 0. The blocked count goes on from where it had dropped to (1), so the full 18 ticks have to pass again before `push`.
- Pressing the arrow again into the wall after letting go starts the count at 1, with no push straight away.

Lifting, carrying, hiding and throwing [live]

Test objects: a bush, a pot, a light stone, a black stone and a sign in a row (`lab1.nw`, y = 20).

- **Lift.** A at the bush (or the pot) plays `lift` at once. A five-step counter runs 4 down to 0, and the carried flag is set on the 5th tick. `carrystill` follows on the **7th tick** (6 ticks of `lift`, as `lift.gani` = 1 frame with `WAIT 5`). The bush tile turns into its stump at once (`arrays.dat` array 12).
- **Carry.** Walking while carrying plays `carry` (8 frames, one per tick) at the normal 0.5 tile per tick. There is no slow-down. Stopping plays `carrystill`. The object is drawn over the head. In the video the head and arms are hidden behind it (`205-pot-throw.png`, first frame).
- **Hide** (D while carrying a bush). In the same tick the pose becomes `hiddenstill`, and the player's "hidden" flag (STATUS bit 0x02, see `carry-hide-and-glove.md`) is set. Only the bush is drawn, where the player stands. Body, head and shadow are all gone (`205-hidden-under-bush.png`).
  - Walking while hidden plays `hidden` at **0.1 tile per tick**. That is a fifth of the walk speed, which confirms the 0.2 factor in `carry-hide-and-glove.md`. The gani frame does not advance: `hidden.gani` is one frame with `WAIT 4` and a step sound.
  - Stopping goes back to `hiddenstill`.
  - D again leaves the hiding place, back to `carrystill` in the same tick.
  - S while hidden throws the bush. The player is in `idle` in that tick, with the hidden flag cleared and the carried flag cleared.
- **Throw** (A while carrying). The player is in `idle` in the same tick. Video of a pot thrown upwards (`205-pot-throw.png`, one frame per tick):
  - The object flies about 6 tiles in 9 ticks. Its shadow moves along the ground while the object itself is drawn rising and then falling back onto the shadow.

    - It breaks into four pieces that spread out over 4 ticks and then vanish.
    - The pot's base tile (the grey plinth) stays where it was lifted.
- A at a black stone with glove 1 does nothing: no lift and no grab. The stone is not a wall tile.

Sword [live]

- S plays `sword` in the next tick. Five frames of one tick each, then `idle` (`sword.gani`: 5 frames, `SETBACKTO idle`).
    - The player does not move while it plays, even with an arrow held.
    - With the arrow still held, walking resumes in the tick after the swing, from walk frame 0.
    - Holding S does not swing again. Every swing needs a new press.
    - Holding S for 2.5 s gives no charge and no spin, with or without the spin status. See the next section.
- **What it looks like** (facing up, `205-sword-up-bush-cut.png`):
    - The blade sweeps from the right side over the head to the left.
    - The bush in front is cut on the **first** swing frame. The leaves fly up and out for about 8 ticks, and the stump is left behind.
    - The shield stays drawn on the right during the swing.
- **Offline editor play with no sword.** A player started from the editor has sword power 0. S still plays the full `sword` gani and sound, but no blade is drawn and nothing is cut. Give the player a sword (`setsword sword1.gif,1` in a level NPC) to test cutting.

Spin attack [live, online]

Recorded on the online lab: the stub server sent `STATUS` (prop 18) = 0x54 (male, weapons allowed, has spin). The offline editor has no way to set the has-spin status.

- **It is a second press, not a charge.** In 2.05 the spin starts when S is pressed again while `sword` is still playing. Holding S down does nothing extra.
    - The decompile agrees: the S handler checks "already in the sword mode (6) and has spin", then switches to the spin mode (13) instead of starting a new swing.
    - The client only sees a new press if S was up for at least one tick in between. A release and re-press inside the same 50 ms tick is missed.
    - A press on swing frames 2, 3 or 4 all worked. A press after the swing has ended (the player is back on `idle`) starts a normal swing, not a spin.
    - A press during the spin is ignored. It does not queue another spin or a swing.
- **Timing.**
    - The swing is cut off: `spin` frame 0 shows in the tick after the second press, so a press on frame 3 skips sword frame 4.
    - `spin` is 13 frames of one tick each (650 ms), then `idle` in the next tick.
    - The client stores the facing direction when the spin starts.
    - The client sends `GANI=spin` to the server once at the start and `GANI=idle` at the end, and asks for `spin.gani` the first time.
- **What it looks like** (`205-spin-attack.png`, facing down; first row is the swing, then the spin):
    - The blade sweeps once round the player with a pale blue trail, starting from the facing direction and turning down, right, up, left and back to down.
    - The body turns with the blade, so the back view shows when the blade is above.
- **2.22 does the same** [2.22]. It was run online against the patched GServer-v2 beta4 lab server, with `STATUS 84` in the account file.
    - A second press on swing frame 3 cut the swing off, and `spin` ran 13 × 1 tick, then `idle`.

    - Holding S gave one swing only, and a third press during the spin was ignored.
    - The 2.22 pack's own `spin.gani` looks the same, blade trail included (`222-spin-attack.png`).
    - A brand-new account created with `STATUS 84` once stalled on "Loading account…"; an existing account edited to 84 logged in fine. Not chased further.
- Not checked: which bushes the spin cuts and whether it hits players or NPCs.

Hurt, knockback and blinking [live]

The test NPC runs `if (playertouchsme) { hurt 1; }`.

- Touching it plays `hurt` in the same tick and takes half a heart (3 → 2.5).
- **Knockback.** The player moves **1 tile per tick for 5 ticks**, along the direction from the NPC's image centre to the player's centre. In the run it went from (29.5, 48) to (28.79, 52.95), which is 5.0 tiles. This is exactly the decompile's description (`decomp-findings-2026-10-25-k/01-knockback-and-packet-hits.md`).
    - If the player is inside a blocking NPC's image, every knock step is refused and the player does not move. A large image, such as `bomb1.png` at 128 × 320, makes this happen.
- **Pose.** `hurt` is shown for **31 ticks** (`hurt.gani`: 1 frame, `WAIT 30`), then `idle`. The hurt pose keeps the facing.
- **Can't be hurt again** for 40 ticks (2 s) from the hit. The second touch in the run came only after that.
- **Blinking.** For the whole 40 ticks the player's body is drawn in one tick and left out in the next: only the shadow shows on the "off" ticks (`205-hurt-knockback-blink.png`). It goes on after `hurt` has given way to `idle`.
- **Walking again with an arrow held.** The hurt counter starts at 40 and drops by one each tick. While it is above 29 the arrows are ignored. From the 12th tick after the hit (counter 28), a held arrow walks again and `walk` replaces `hurt`. So `hurt` lasts 31 ticks only when no arrow is held. Re-run 2.05 with Left held through the hit: hurt at tick 88, knockback ended at tick 93, walking at tick 100. Blinking and the 40-tick no-rehit go on regardless.

Swimming [live]

Water tile 461 (passable, liquid); the pool starts at x = 40.

- The switch to `swim` happens on the tick where x + 1.5 reaches 40 (x = 38.5). Leaving the water happens where x + 1.5 drops below 40 (x = 37.5). In that tick `walk` comes back, at frame 0.
- Swimming speed is the same 0.5 tile per tick as walking.
- `swim` steps one frame every **2 ticks** (8 frames, `WAIT 1`), so the stroke takes 0.8 s.
- **It keeps animating while the player is not moving.** It does not reset when the player turns.
- The splash at entry and exit is in `water-and-ganis.md`.

Grab and pull [live]

At a wall of deep-water tiles, facing down:

- A at the wall plays `grab` at once and holds it.
- Holding the arrow away from the wall (Up) plays `pull`: 3 frames of 2 ticks each (`pull.gani`, `WAIT 1`). The pull cycle counter goes 2 … 19 and back to 2, so it repeats every 18 ticks, as in `ledges-push-pull.md`. Nothing moves against a wall.
- **Letting go of the arrow while still holding A.** The cycle counter drops to 1, but the pose stays `pull` and keeps animating. It does not go back to `grab`.

- **Letting go of A as well.** The pose still stays `pull`, animating in place, until the next key moves the player. This answers the open question in `ledges-push-pull.md`: nothing sets `idle` there, and the stale pull loop is what is drawn.

Other builds: 1.39, NewMain 1.41 and 2.22

How each was run

- **1.39** is the last client before ganis, and its player animation is built into the executable. It runs in its own One Player mode, which loads `levels/start1.graal`.
  - The test level was rewritten as a `GR-V1.03` file with the same objects (`mkgraal.py`). The objects are moved so that the view does not clamp at the edges: the row of five at y = 24, the water at x 42 to 59, the wall at y = 44, and the bushes at y = 34.
  - The player starts at (21.5, 28).
  - Keys reach the game only with `nodirectinput=true` and `joystickon=false` in `graalconfig.txt`.
- **NewMain 1.41.1** (`graalnm24`, January 2001) is the first gani client. It is an update pack: copy it over the 1.39 folder, because it needs 1.39's `headsconfig.txt` and GIF heads. It opens a `.nw` in the editor and plays it the same way as 2.05 (`startnw.sh`).
- **2.22** (`Graal_222.exe` in the Reborn folder) also opens in the editor. Its Load dialog did not take the typed start position, so the player started at the default (30.5, 30).
- **Finding the player record.** In each build the record starts with its class pointer, followed by x at +4 and y at +0xC (doubles), as in 2.05. One global points to it. `diffscan.py` finds it by walking a known distance; it checks doubles at both 4-byte alignments.

Cross-version table

"=" means the same as 2.05.

behaviour	**1.39**	**1.41**	**2.05 rev 2**	**2.22**
walk speed / frame cycle	0.5 per tick, 8 ticks [video]	=	0.5 per tick, 8 frames × 1 tick	=
blocked count: push at 18, wrap 38 → 18	= (the same counter)	=	yes	= (push in the tick after 18 blocked ticks)
push pose frames	about 2 ticks each [video]	4 ticks (`push.gani`)	4 ticks	= (5 × 4 ticks, looping)
lift	counter 4 → 0, carried on the 5th tick; no lift pose of its own	`lift` 6 ticks, then `carrystill`	=	=
carried-object id	261 for a bush (the same sprite number)	–	261	–
hide (D while carrying)	hidden flag; walk 0.1 per tick	hidden flag, but the pose **stays `carrystill` / `carry`**, with the legs cycling at 0.1 per tick	`hiddenstill` / `hidden`	= 2.05
S throws from hiding	yes	yes	yes	yes
S throws while carrying (not hidden)	–	–	yes [dec] (not traced live)	**yes**: `carrystill` to `idle` in the tick after the press, no `sword` frame
sword	counter 1 → 6 (5 ticks); cuts the bush	`sword` 5 ticks; cuts the bush on the first frame (`141-sword-up-bush-cut.png`)	=	–
hurt: lock-out / no-rehit	40 ticks	40	40	=

behaviour	1.39	1.41	2.05 rev 2	2.22
knockback	**4 steps** of 1 tile (4 tiles)	5 steps (5 tiles)	5 steps	= (4.9 tiles on the same test)
hurt pose	about 10 ticks, then the normal pose [video]	`hurt` 31 ticks	31 ticks; 12 with an arrow held	= (12 with an arrow held)
blinking	starts **only after the hurt pose**, then every other tick until tick 40 [video]	from the hit, every other tick	from the hit	–
swim entry / exit point	x + 1.5 on liquid	=	=	=
swim speed	0.5 per tick	=	=	=
standing still in water	the water ring keeps alternating; the head stays still [video]	**`swim` frame resets to 0 and freezes**	keeps animating	keeps animating; the frame resets to 0 at the stop, then runs on
letting go of the arrow while pulling	–	back to **`grab`**	stays `pull`, looping	stays `pull`, looping
letting go of A after a pull	–	`grab` stays until the next action	`pull` loops until the next move	= 2.05

Notes per build

- **1.39 hurt** [1.39]. The touch NPC uses the same `hurt 1`. The pose shows the player's back with the arms out (`139-hurt-then-blink.png`).
- **1.39 sliding past a blocking NPC** [1.39]. Walking right into a character NPC's side nudged the player along it by 0.125 tiles per tick. The player got around it, and `playertouchsme` did not fire on that contact. A timer NPC that tests `playerx` / `playery` was used for the hurt test instead.
- **1.39 swim** [1.39]. Only the head and a water ring are drawn. Standing still, the ring alternates between two shapes (`139-swim-standing.png`).
- **1.41 hide** [1.41]. The `graalnm24` gani folder does hold `hidden.gani` and `hiddenstill.gani`, but the 1.41 client does not play them. The hidden player keeps the carry poses, and the leg cycle still runs one frame per tick at a fifth of the speed.
- **1.41 sword** [1.41]. With `setsword sword1.gif,1` the blade is drawn and the bush in front is cut on the first swing frame, as in 2.05.
- **2.22 sword while carrying** [2.22]. In the `t222` run the bush is lifted, D hides, the player walks hidden, D comes out (`carrystill`), then S. The tick after S shows `idle`, with no `sword` frame: the sword key throws, as the 2.05 key handler does. 2.31 was not run. Not run in any build: S during the six `lift` ticks, and a throw with an arrow held.
- **2.22** [2.22]. Lift, hide, the stuck pull loop, swimming in place, push and hurt all behave as in 2.05. The hurt test was the same walk into the touch NPC from the right, with the arrow held, and gave the same knockback and the same 12 ticks before walking resumed. The only difference seen is that `swim` restarts at frame 0 when the player stops.

5.x and 6.x Windows clients

`protocol/v5v6-lab-play.md` has the same tests on 5.24r4, 5.30r3 and 6.03r7, timed from their own messages on a lab server. Walk 0.5 tile per tick, `push` at about 0.9 s, `lift` for 0.3 s then `carrystill`, throw in the same tick, and `sword` for 5 ticks with the bush cut as the swing starts are all the same as 2.05. New in those builds: while pushing, the client sends the trigger action `pushed,0` once a second. Chat is outlined text with no bubble, and it is cleared by the client after about 3 s. Hiding under a bush or stone (status bit 0x02, only the object drawn, 1.5 px per tick) and the pot's "D does nothing" also match 2.05. Grab and pull start in the next tick, and the client sends `pulled,1` 0.85 s into a pull. A hit plays `hurt` for 1.55 s, which is the 31 ticks of the `hurt` gani below, and pushes the player 5 tiles in 5 ticks. Swimming is as fast as walking.

The built-in ganis across the gani sets [gani]

The 2.0x archives in the project hold only the executables, and the 2.05 client fetches its ganis from the server. The gani sets on hand are:

- NewMain 2.4 and 2.6, the first gani clients (December 2000 files);
- NewWorld beta 1 (August 2001);
- the Graal Reborn 2.22 pack.

NewMain 2.4 and 2.6 are identical. In the table below, frames × ticks gives the ticks per frame (1 + WAIT); ">x" is SETBACKTO x; L means LOOP.

gani	NewMain 2.4 / 2.6	NW beta 1	2.22 pack
idle	1	1	1
walk	8 × 1, L	8 × 1, L	8 × 1, L
walkslow	8 × 2, L	–	8 × 2, L
sword	5 × 1, >idle	5 × 1	5 × 1, >idle
spin	13 × 1, >idle	–	13 × 1, >idle
push	5 × 4, L	**3 × 4**, L	5 × 4, L
pull	3 × 2, L	3 × 2, L	3 × 2, L
grab	1	1	1
lift	1 × 6, >carrystill	**6 × 1**	1 × 6, >carrystill
carry	8 × 1, L	8 × 1, L	8 × 1, L
hidden	1 × 5, L, single direction	(no frames)	1 × 5, L, single direction
kick	1 × 5, >idle	1 × 5	**1 × 11**, >idle
shoot	1 × 11, >idle	1 × 11	1 × 11, >idle
hurt	1 × 31, >idle	31 frames (10 × 2 then 21 × 1)	1 × 31, >idle
dead	**13 × 4** (52 ticks)	5 frames, >gameover	**15 frames** (3 × 7 + 2 × 8 = 37 ticks)
swim / lava	8 × 2, L	8 frames (6 × 2 + 2 × 1)	8 × 2, L
ride	4 × 2, L	4 × 2, L	4 × 2, L
sleep	1	**2 × 19**, L	1

- Every 2.05 live timing above matches the NewMain / 2.22 column: walk, push, pull, lift, carry, sword, hurt and swim.
- **The hidden ganis.** In NewMain, `hidden` and `hiddenstill` draw only a shadow, and the client itself draws the carried object. The 2.22 pack adds the object to the gani as an `ATTR3` sprite (`bushitem.png`). A 2.22 server's ganis drawn by the 2.05 client would therefore show the bush twice.
- Every set keeps the same sounds on the same frames: `steps2.wav` on walk frames 0 and 4, `sword.wav` on frame 0, `lift.wav` on lift and hurt, and `water.wav` / `water2.wav` on swim / lava.
- The full per-file output, with frame-content hashes, is `gani-compare.txt` in the project folder.

Asset gotcha: 2.05 loads only GIF heads, bodies and swords [live]

Given the 2.22 folder, where heads, bodies, swords and shields are PNG, 2.05 rev 2 looks only for `head0.gif` and the like. It finds nothing and draws the player with no body. That is the "missing body" in `live-trace-205.md`, and it is not a Wine bug. Converting `levels/{heads,bodies,swords,shields}/*.png` to `.gif` next to them fixes it. A Seihai 2.05 profile that resolves player images should try `.gif` only.

What Seihai does differently (from its `client/src/pose.rs`, 2026-10-30)

Seihai was not changed here. These are what a 2.05 profile would need:

1. **Push.** Seihai shows `push` after 17 blocked steps and advances it one frame per blocked step. The real client shows it on the 18th and holds each frame for 4 ticks (it honours `WAIT`).
2. **Pull.** Seihai steps `pull` once per tick; the real client steps it once every 2 ticks. Seihai drops the pose on release; the real client leaves `pull` looping.
3. **Gani WAIT.** Seihai does not read per-frame `WAIT`. The real hurt pose is 31 ticks with no arrow held, or 12 ticks before a held arrow walks again (Seihai uses a 500 ms minimum). Lift is 6 ticks. Swim is 2 ticks per frame, which Seihai already uses.
4. **Swim while still.** Seihai steps `swim` per step taken. The real client keeps it animating when the player stands still in water.
5. **Hurt blink.** There is no blink in Seihai. The real client draws the body on alternate ticks for 40 ticks.
6. **Hide.** There is no `hiddenstill` / `hidden`, no 0.1 tile per tick walk and no hidden flag in Seihai.
7. **Lift pose.** Seihai goes straight to `carrystill`. The real client plays `lift` for 6 ticks first.
8. **Spin.** Seihai records the `spinattack` flag but never plays `spin`. The real client turns a second S press during a swing into a 13-tick spin.

Harness

- `act.py <name> "<keys>"` runs a key script, for example `down:Up w:0.6 up:Up tap:a`. It records the player state (gani, frame, position, every changed field) and 60 fps video at the same time. A `tap` holds the key for 120 ms: shorter taps are lost between ticks. `NOMEM=1` records video only, for builds whose player record has not been mapped.
- `sh.py` prints a trace, with the key presses interleaved.
- `mkgraal.py` writes the 1.39 test level as `GR-V1.03`, with NPCs given on the command line. `start139.sh` starts 1.39 in One Player mode, and `startnw.sh <dir> <exe> <level> <x> <y>` starts an editor-era build on a `.nw`.
- The per-build settings for `act.py` (process name, player-record global, gani offset) are in `README.md` in the project folder.
- `act.py` also takes `gx:` / `gy:` steps, which walk until the player reaches that x or y.
- `mklevel.py` writes `lab1.nw`:
  - five liftable objects in a row at y = 20;
  - a pool of water tile 461 at x 40 to 55, y 10 to 30;
  - a deep-water wall at y = 40;
  - four bushes at y = 50;
  - a `setsword` NPC, a touch-to-hurt NPC at (30, 44) and a `lay spinattack` NPC.
- `gani.py` summarises and compares gani folders.

Open

- **The spin attack in 1.39 and 1.41.** It was seen only in 2.05 and 2.22, online. Offline, neither `lay spinattack`, `set canspin` nor a `spinattack` chest gave the has-spin status.
- **1.39's push and walk frames** are not in its player record as plain numbers; they were read from video only.

`client-internals/precursor-runs-2026-10-30.md`

Precursor runs and behaviour constants (2026-10-30)

This note follows `precursors-and-fan-builds.md`. That note was a static read. This time the Java precursors and the Game Boy Color demo were **run** in a sandbox, and the game-feel numbers were read out of the Zelda Online 2 and Java Graal sources. The aim is to give Seihai's per-version profiles exact numbers for the oldest builds: tick rate, walking, pushing against walls, lifting, bush drops, respawn, and how chat is drawn.

Tags: **[src]** read from the Java source, **[live]** seen in a running client or on the wire. No code is quoted. Screenshots are in the project's shared folder under `graal-decompile/precursors/runs-2026-10-30/`, not in this repo.

How it was run

- **Zelda Online 2.** The bundled GServ Standalone server ran on port 14801, with two copies of the applet in an applet harness under Xvfb. Both clients joined, saw each other, walked, cut bushes, pushed against walls and chatted. The traffic was captured with tcpdump. **[live]**
- **Java Graal (July 2000).** Two copies of the applet ran in the same harness. The applet only connects to its own codebase host on 14802, so a plain TCP forwarder from 14802 to the GServ server on 14801 stood in for the old relay. Both players saw each other. **[live]**
- **Game Boy Color demo.** Run in PyBoy with scripted input. **[live]**

Behaviour constants (Zelda Online 2 and Java Graal)

Every number below is the same in both games. The Java Graal applet is the Zelda Online 2 engine, re-skinned with Graal art and given new levels.

What	Value	Tag
Main loop	50 ms per tick (20 Hz)	[src]
Walk step	half a tile per tick while a key is held	[src]
Corner slide (walking into the edge of a block)	a quarter of the walk step, sideways around the corner	[src]
Sword	5 ticks; hits are tested on ticks 1 to 3 only	[src]
Being hurt	40 ticks of hurt state, half a heart of damage	[src]
Shield bump	4 ticks	[src]
Lifting	5 ticks	[src]
Baddy moves	sent by the level leader every 2 ticks (100 ms)	[src]

Walking into a wall

The player keeps a push counter that counts ticks spent walking into something solid. **[src]**, and the pose was seen in the running client. **[live]**

- **Ticks 0 to 17 (0.9 s):** nothing visible. The player stands in the walk pose against the wall.
- **Tick 18:** the push pose starts. The push frames advance one step every 4 ticks. The ledge-jump check also runs from tick 18, so a jump down a ledge needs 0.9 s of pushing.
- **Tick 28 (1.4 s):** a push stone in front of the player moves.
- **Tick 38:** the counter goes back to 18, so the push animation loops without restarting the 0.9 s wait.
- Letting go of the key, or turning, resets the counter to 0.

Compare `ledges-push-pull.md` for the Delphi clients.

Bushes, stones and pots

- **Cutting a bush** rolls one number from 0 to 99. **[src]**
  - 0 to 24: one green rupee (25%).
  - 25 to 29: a heart (5%), worth one heart of power.
  - 30 and 31: a five-rupee item (2%).
  - 32 to 99: nothing (68%).
- **Lifting.** There are four liftable object kinds. Kinds 0 to 2 can always be lifted. Kind 3 (the heavy stone) needs the stronger glove. A lifted bush (kind 0) rolls the same drop as a cut one. Kind 2 always leaves a heart. **[src]**
- **While carrying**, the player is drawn with a separate set of four carry sprites, one per direction, with the object drawn above the head. The player is not hidden under the object in these games. Hiding under a carried bush came later; see `carry-hide-and-glove.md`. **[src]**
- **Respawn: there is no timer.** Cut bushes, lifted stones and broken pots stay gone only for the visit. Every board change is cleared when a level loads, so everything is back when the player leaves and comes back. Nothing grows back while the player stays on the level. **[src]** Re-entry was not tested live.
- **Sharing board changes.** Each change is relayed as a `B` line. The level leader (earliest login time) sends all of the current changes to every newcomer, so a player who joins sees the bushes already cut. **[src]** The captured sessions had no board changes on the wire, because nothing was cut while both clients were in the same room. **[live]**

Chat

There is no chat bubble. [src], [live]

- Chat text is drawn in TimesRoman bold, 20 points, in black, over a white copy of the same text shifted one pixel right and one pixel down. The result reads as black text with a thin white edge.
- The text is centred on the player (the player's x plus 12 pixels, times the zoom), just above the head.
- The nickname is drawn in the same style three tiles lower, under the feet.
- There is no box, no background and no tail. The bubble seen in later Graal belongs to the Delphi clients, not to these.

What the wire showed

- **Room names.** Zelda Online 2 joined as `NZlevel11.6#def`, and Java Graal joined as `NG2level13.graal#def`. These match what the sources say. **[live]**
- **Join order.** The server answers `I<id8>`, then sends `+<id8><name>` for each available player. The client answers the first `+` with `P<id8>` and then pushes its whole state: login time, position, chat, nickname, colours and carried object. **[live]**
- **Doubled relay.** When two clients see each other at the same moment, they both send `P` for the other. GServ then links the pair twice, and every `D` line after that is delivered twice. One capture had 1144 relayed lines for 572 sent. Nothing visibly broke, because most `D` lines are state updates such as position (550 of 578 sent lines in the Zelda Online 2 capture were `M` moves). A doubled one-off event, such as a board change, would simply be applied twice. It happened with both Zelda Online 2 and Java Graal. A server that emulates GServ for these clients does not need to reproduce it. **[live]**

Game Boy Color demo in an emulator

This adds a run to the static read in `gameboy-demo.md`. Everything it says about the screens held up. **[live]**

- There is no title screen. The ROM goes straight from the Nintendo logo to room 0, the outdoor start with palm trees, bushes and flowers.
- **Select** opens the character carousel in the window. It shows three heads with the middle one's name in the font, and the item slots labelled "Button B" and "Button A" under it. Turning it left and right went through Vincent, Fung Si Yun, Pach Soldier, Ninji, Lunar and Squall in that order, and each step changed the player's look straight away. This confirms the static finding that the carousel is a free picker over all character types.
- The item icons under the slots change with the character. For example Fung Si Yun shows a green item in slot A, and Pach Soldier shows a blue sword in slot B.
- Walking around room 0 and the next rooms showed the NPCs and log walls where the room table in `gameboy-demo.md` puts them.
- Start (music) was not checked by ear here. A later, longer run (see "Seen in an emulator" in `gameboy-demo.md`) found that Start mutes the game.

What this means for Seihai profiles

For a "Zelda Online 2" or "Java Graal" profile:

- Run game logic at 20 Hz and move half a tile per tick.
- Show the push pose after 0.9 s against a wall, and move push stones at 1.4 s.
- Use the bush drop table above.
- Never regrow anything during a visit, and restore everything on level load.
- Draw chat as bold serif black text with a white one-pixel offset and no bubble.
- Do not hide the player under a carried object.

For the Delphi Graal builds these numbers are a starting point only. Their wall push, hiding and respawn timers have their own notes and are not covered here.

Open

- Whether the 1st to 4th Delphi betas still used the 0.9 s / 1.4 s push timings. The Delphi code has not been compared yet.
- The GB demo's music format, which depends on cartridge RAM that the ROM never writes. In the emulator, Start only muted the game (see `gameboy-demo.md`).

`client-internals/precursors-and-fan-builds.md`

Precursors and fan builds

Date: 2026-10-29

This note covers a set of uploads that sit around the Delphi client line:

- the Zelda Online Java versions (1998)
- Stefan Knorr's Zelda level editor (1998)
- Java Graal (2000)

- the Graal Game Boy Color demo (1999)
- 39ster's Graal client 1.00 (2014)
- GraalX (2001)

The sources were read where they exist and the class files and binaries were read statically where they don't. Nothing was run. No code is reproduced here. Two of the "source" archives turned out to be decompiler output and are treated that way (see below).

What the uploads are

There are 14 uploads but only 9 distinct items. They were compared by content hash.

Upload(s)	Same as	What it is
`zeldaonline.zip`, `zeldaonline1.rar`	each other, except that the rar adds 36 levels	Zelda Online 1: a single-player Java applet
`zeldaonline1_src.zip`	–	Decompiler output for the Zelda Online 1 classes, made in 2003, not the author's source
`zeldaonline2.zip`, `zeldaonline2.rar`	each other, byte for byte. The zip is also the copy in `zeldaonline2.7z` in the project files	Zelda Online 2 "local version": game, server and the author's commented source
`zeldaonlineeditor.zip`, `zeldaonlineeditor.rar`	each other. The exe is also in the next item	The Zelda level editor (`ZeldaEditor.exe`)
`ZeldaOnline_aka_very_first_graal.zip`	–	The editor, the untouched `zeldaonline2.zip`, and an unpacked Zelda Online 2 that a player edited in February 2002
`javagraal_rip.rar`	–	Java Graal as served by Cyberjoueur: `Graal.jar`, loose classes, art and 261 level and asset files. The archive is damaged: 26 files (10 `.graal`, 15 `.zelda`, 1 more) fail extraction
`javagraal_src.zip`	–	Decompiler output for the five classes in `Graal.jar` (2003). Its class and method set matches the jar exactly
`graalgb.rar`, `Graal v1.10 - Adventure Demo (PD) [C].zip`	each other: the same 256 KB ROM	The Graal Game Boy Color demo
`GraalX.rar`	–	A 2.01-era Delphi client. Already in `build-catalogue.md`
`39ster Graal v1.00.7z`	–	A 2014 fan remake. It is not the 39ster 2.1 GServer covered in `ash/`

How they relate

```
1998-05  Zelda Online 1      Java 1.0 applet, single player, Z3-V1.02/1.03 levels
1998-07  Zelda Online 2      Java 1.1 app, adds multiplayer through GServ on 14801
1998-07  ZeldaEditor         Delphi 3, writes Z3-V1.03; its dialogs become 1st Graal's editor
1998-09  1st Graal           Delphi; reuses the editor forms and the relay lobby
1999-09  Graal GB demo       Game Boy Color homebrew by Knorr ("Graal v1.10 Adventure Demo")
2000-07  Java Graal          Zelda Online 2's client with Graal art, reading GR-V levels, relay on
14802
2001-03  GraalX              2.01 pre-release that keeps the .nw loader
2014-04  39ster client 1.00  offline fan remake on an SDL/GameMonkey engine
```

Zelda Online 1 (May 1998)

- A Java 1.0 applet (`Zelda` class, class file version 45). The page runs it at scale 2 in a 512×448 box. The class files are dated 16 May 1998.

- **Single player only.** There is no socket code, and levels, `arrays.dat`, images and sound are all fetched relative to the applet's codebase.
- The start level is `level1.zelda`. The rar ships 36 levels, all `Z3-V1.03`. The loader also accepts `Z3-V1.02`.
- The story is hard-coded per level name: people placed on `level13`, `level28`, `level22` and `cave2`, and chests on `level21`, `level24`, `village3`, `level16` and `level29`, all gated by progress flags.
- The class already holds the text alphabet that later Graal uses for sign text (the 97-entry string documented in `formats/sign-text.md`), together with the glyph-code list `A B X Y u d l r h x y z 1 2 3 4 ..`. So the Graal sign alphabet goes back to the first Java applet.

Zelda Online 2 "local version" (July 1998)

- A Java 1.1 *application*, not an applet. It opens a start window with "One-Player mode" and "Multiplayer mode" and a server address field that defaults to the loopback address. It refuses to run on Java 1.0.
- Files are read from the working directory, not over HTTP.
- The package has `game/`, `server/` and `programming/` folders, and the author's real source is included, with comments. The files are dated 18–26 July 1998.
- It ships 51 `Z3-V1.03` levels. `level1.zelda` is again the start level.
- There is an extra `ZeldaMain (1).class` at the top level. It differs from `game/ZeldaMain.class` in one constant only: the tile index of the "free stone" (the tile left behind when a stone is lifted) is 1023 instead of 584.
- The options window has a nickname field, an option to show your own name, sprite colours for cap, cap edge, belt and coat, and a **password**. The password is a 10-character save code (see below).
- The readme warns that the local version uses different tile indexes from the online server, so bushes and stones look wrong when the two are mixed.

Password saves

The save code is 10 characters from a 64-character alphabet, so it holds 60 bits. Packed into it are:

Field	Bits
player x (0–63)	6
player y (0–63)	6
level index (from a fixed level list)	9
rupees	10
darts	6
power, in half hearts	5
glove level 2, can shoot, has map, found brother	1 each
the four story chests opened	1 each
checksum: the number of set bits in the payload	6 (lowest)

The 60 bits are then XORed with a fixed mask and their positions transposed (bit *i* moves to `(i mod 10)·6 + i div 10`) before being written out six bits per character. Reading a code reverses those steps, checks the bit count, and fails on a wrong length or an unknown character.

Zelda Online 2 in the "very first graal" zip

The `ZeldaOnline2` folder in `ZeldaOnline_aka_very_first_graal.zip` is the stock release plus a player's edits from 13–15 February 2002:

- 4 new levels (`agrethouse1`, `agretweapons1`, `eggsecretroom1`, `gameover`)

- 4 changed levels (`level1`, `level11`, `level29`, `house15`)
- an `objects.dat` with 4 more stamps (109 instead of 105): "Chest - Red Rupee", "Blackness", "Big Blackness" and "Breakaway Wall Part"
- a `GServProto.txt` log line from one server start

None of this is from 1998.

ZeldaEditor (July 1998)

- Built with Delphi 3 (Borland RTL, "Copyright (c) 1983,97"). Linked around 25 July 1998.
- Main window `TZeldaFrame`, titled "Zelda Editor". It has a toolbar with New, Open, Save, Save As, Undo, Cut, Copy, Paste, Delete, "Turn Selection Into A Warp Field", Edit Links, Edit Baddies and Edit Signs. A stamp ("object") palette sits on the right with add, save and delete buttons.
- Dialogs: `TLinkList`, `TLinkOptions`, `TNewOpponent`, `TEditSigns`, `TObjectName` and `TShowCodes` (the sign glyph codes).
- It saves only `Z3-V1.03`. Window positions are kept in `ZeldaEditor.ini`, one `Name:left,top,width,height` line per window.

1st Graal grew out of this editor. The 1st Graal client (Sept 1998) carries `TEditSigns`, `TLinkList`, `TLinkOptions`, `TObjectName` and `TShowCodes` under the same names. `TEditSigns` differs only by a few pixels plus `KeyPreview` and a key handler. `TNewOpponent` became `TEditBaddy`, and the main form became `TAdventure`. So the Delphi Graal client started as the level editor with a game added, not the other way round. This fits Graal shipping with an editor built in.

`objects.dat` (stamps)

The editor's stamp list is a flat sequence of records:

- the name, ended by `\n`
- one byte for the width and one byte for the height
- width × height 32-bit little-endian tile ids, row by row

Tile id 2047 (`0x7FF`) is a transparent cell. The 1998 file has 105 stamps, from "Bush", "Sign", "Stone" and "PushStone" up to whole houses, bridges and a cathedral. The same format is still used by the `objects.dat` shipped with Graal 1.21.

Shared Java game data

`arrays.dat` is built by the `makeArrays` and `addSprites` tools in `programming/`. It is a list of typed integer arrays. Each array starts with a 32-bit big-endian header: the high 16 bits give the kind (0 for a flat int array, 1 for an array of arrays) and the low 16 bits the count. The values that follow are big-endian 32-bit ints.

The game reads these arrays in this fixed order:

1. non-wall tiles
2. swamp tiles
3. jump stones
4. bush objects and their replacements
5. liftable objects and their replacements
6. chest tiles and their replacements
7. movable stone

8. sprite crop table
9. hurt sprites
10. leap animation
11. baddy sprites
12. bumped sprites
13. explosion frames
14. swamp-soldier frames

Java Graal and 39ster's client also ship an `arrays.dat`. 39ster's is a plain-text tile-type table instead.

Level format as the Java code reads it

This confirms and extends `formats/level-binary.md`:

Magic	Code width	Read by
`Z3-V1.02`	**11 bits**	Zelda Online 1 and 2 only
`Z3-V1.03`	12	Zelda Online 1 and 2, ZeldaEditor (writes it), Java Graal
`Z3-V1.04`	12	Java Graal
`GR-V1.00`, `GR-V1.01`	12	Java Graal, 39ster
`GR-V1.02`, `GR-V1.03`	13	Java Graal, 39ster

- The run-length rules are the ones already in `level-binary.md` (control flag in the top bit, count in the low byte, `0x100` for a double run). A decoder that follows them reads all 271 levels in these uploads with no errors.
- In `Z3-V1.02` and `Z3-V1.03` the Java games read each baddy as a fixed 4-byte record: x, y, type, then a byte that is always `\n`. That is the same thing as "3 bytes plus an empty verse line". The list ends at a record whose first byte is `0xFF`.
- Link lines are `file x y w h newx newy`. A negative `newx` or `newy` means that coordinate is kept when warping.
- Sign lines are x+32, y+32, then text in the sign alphabet.
- Java Graal reads GR-V NPC lines but only shows the image: the script after `#` is dropped, and a missing `.gif` suffix is added. It skips the GR-V1.01+ chest section.

Network: GServ Standalone and the Java clients

Zelda Online 2's `server/` holds "GServ Standalone (C) Stefan Knorr 1997, based on JccVsd (C) Stephan Kulow". It is a game-neutral lobby that Knorr also used for chess (a `ChessSync` thread sends every "Chess" player a one-character timer line every second). It listens on **14801**, gives out ids `0000` plus a 4-digit counter that wraps at 9999, and logs to `GServProto.txt`. This is the lobby that `protocol/relay-1998.md` describes from the Delphi side. The source adds:

Line	Direction	Meaning
`N<game>#<name>`	C→S	Join as a player (state "waits") and get `I<id>`
`O<game>#<name>`	C→S	Join as an observer. You see players in that game who are already in a party
`S<name>`	C→S	Super user: sees every player in every game, listed as `name (game)`
`+`	C→S	Available again (after `-`)
`-`	C→S	Not available. Your id is withdrawn from the other players' lists

Line	Direction	Meaning
`P<id8>`	C→S	Join that player's party. You are linked to them and to everyone already in their party
`C`	C→S	Leave the party
`V<id8>`	C→S	Watch a player: you receive their party's `D` traffic
`D<data>`	C→S	Relay to everyone in your party
`H<score>`	C→S	High score. The client sends it and this server ignores it
`I<id8>`	S→C	Your id
`+<id8><name>`	S→C	A player in your game is available. The name follows the id directly
`-<id8>`	S→C	That player left or became unavailable
`P<id8>`	S→C	That player is now in your party
`C<id8>`	S→C	That player left your party
`D<data>`	S→C	Relayed data, unchanged. The server does not add the sender

Zelda Online 2 client

- Connects to the typed address on 14801.
- Room name: `Z` + level file name without extension + `1.6`, for example `Zlevel11.6`. Every level is its own room. On a level change the client sends `C` and `-`, then joins the new room with the name `def`. The real nickname is sent in `D` later.
- When the first `+` arrives it sends `P` for that player straight away. On `P` it pushes its state: login time, position, chat, nickname, colours and carried object.
- `D` payload types: `P C N S M H B T D E O W A`. Their layouts are the ones in `relay-1998.md`, and the 1st Graal kept them. The level-leader rule (earliest login time, ties broken by the lower id) also comes from here.
- The client clamps baddy knock-back to ±4 before encoding. One of the four clamps writes the wrong axis, so a large positive vertical knock-back overwrites the horizontal value.

Java Graal client

- Connects to **its own codebase host on 14802**, the Cyberjoueur relay port that the Delphi betas also used.
- Room name: `G2` + level file name. A second "fixed room `Graal2`" mode exists in the class but the applet never selects it.
- `D` types: the Zelda Online 2 set without `S` (colours) and with `L` (level name) added.
- Without `S`, Java players appear with default colours. Delphi clients of the same period send types the applet ignores (`F`, `G`, `I`, `Q`, `R`, `U`, `X` to `Z`, `0` to `8`). So mixed sessions would only share movement, chat, board changes, baddies, items and arrows. This is *inferred*, not observed.
- **The `G2` prefix and the `Graal2` fallback are exactly what 4th Graal alpha (October 1998) uses.** The room prefixes and fallback names were read from the string tables of all 18 betas. 1st and 2nd use `Zelda2`, 3rd and 4th use `Graal` with a bare `G` prefix, and 4th alpha alone uses `G2` / `Graal2`. From 5th Graal on (`G3` / `Graal3` up to `G8` / `Graal8`) the rooms no longer match.
- So Java Graal speaks the relay dialect of 4th Graal alpha. Its `D` types are all ones that 4th alpha also sends or handles, and 4th alpha's extra types (`I`, `U`, `X`, `Y`, `Z`, `Q`, `F`, `G`, `R`) are ones the applet ignores. A Java player and a 4th alpha player on the same level file were in the same room. Later betas were not. This is read from the binaries and the Java source, not from a capture. The only open point is whether 4th alpha appends the bare file name for a local level, as the applet does. **Run live on 2026-10-31:** it does (`G2level13.graal`),

and a Java Graal and a 4th alpha player met and chatted on Knorr's GServ. Java Graal and 5th Graal met through a room-renaming proxy (`relay-room-bridge.md`).
- The applet was built in July 2000, 20 months after 4th alpha, so its dialect was already long out of date. It only made sense on a relay that still had 4th alpha clients, or Java players alone.

Java Graal (July 2000)

- `Graal.jar` holds 5 classes (`Graal`, `GraalClient`, `Computer`, `FileLink`, `ServerPlayer`), all dated 2 July 2000 and signed (SHA and MD5 digests in the manifest). The rip's loose copies are identical to the jar's.
- It is a Java 1.1 applet (512×418) embedded in a Cyberjoueur page next to their Bomberman, chess, Reversi and Tetris applets, with the Flycast ad block from 1999. The rip itself was taken on 23 February 2002.
- In code terms it is Zelda Online 2's game and client with Graal art (`pics1.gif`, `sprites.gif`, `letters.gif`) and Graal levels. It starts on `level13.graal` and has a `nickname:` chat command.
- `levels/` holds 180 readable levels (plus 25 damaged ones), NPC images and a few WAV files. Many `.zelda` files actually carry `GR-V` headers. The level names (`ebon*`, `mod*`, `sominside*`, `dungeona*`) look like player-made Cyberjoueur web levels.

Graal Game Boy Color demo (1999)

The full disassembly is written up in `gameboy-demo.md`. In short:

- The ROM header has the title `GRAAL`. It is dual-mode (it runs on both Game Boy and Game Boy Color) on an MBC1 cartridge with 256 KB of ROM. Only banks 0 to 5 are used. The header checksum is valid and the file is dated 29 September 1999.
- It is a C program built with GBDK 2.x and LCC. The whole game runs in the VBlank interrupt.
- "Welcome to Graal / Demo version by Stefan Knorr" is a sign in the first room; there is no title screen. So this is Knorr's own port, not a fan project. The "(PD)" in the dump name marks it as public-domain homebrew.
- There are seven rooms of 32×32 tiles. Their links use the PC client's link record, with a room number in place of the file name.
- Gameplay text includes a captive you free, who shows you how to lift and throw stones and then tells you to press Select and switch to "Fung Si Yun".
- Names in the ROM such as Lizardon, Pachukite, Pach Soldier, Gold Soldier, Water Athena and Red Squall belong to a table of 31 character types, not only to baddies. The Select menu lets the player switch to any of them.
- The text font is ordered space, A–Z, a–z, 0–9, then punctuation, starting at byte `0x20`. This is not the PC sign alphabet.

39ster's Graal client 1.00 (2014)

- All files are dated 23 April 2014. `GameMaker.exe` is a small C++ engine on SDL 1.2 (plus zlib and stb_image), unrelated to YoYo's GameMaker. It embeds the GameMonkey scripting language, and all of the Graal logic is in `main.gm`, about 1,500 lines of script.
- It reads `GLEVNW01 .nw` levels and `GR-V1.00`–`1.03` levels with the same 12- and 13-bit rule, plus `.gani` animations. It converts tile ids into the 2.x `pics1.png` layout and starts on `level13.nw`.
- **Offline only.** It has no socket code, and NPC scripts are not run: the NPC object has an execute hook with the call commented out, so NPCs only show their images.
- It bundles 1,733 `.nw` levels and 1,321 GIFs collected from community worlds.

GraalX

GraalX is already in `build-catalogue.md`: a pre-release of 2.01, linked 17 hours before it, with 2.01's token, list port and DES-hidden list host, which keeps the `GLEVNW01` reader and writer. A form-by-form comparison adds one more check. GraalX has the same 28 forms as 2.0 rev3 and 2.01, and differs from 2.01 only in two control positions on the main form.

Open

- ~~Whether a Delphi beta ever used the `G2` room prefix.~~ Answered above: 4th Graal alpha did, and no other beta.
- ~~The GB demo's full script and map format.~~ Answered in `gameboy-demo.md`.
- Who made the February 2002 Zelda Online 2 edits.

`client-internals/prior-attempts.md`

Prior attempts to understand the Graal environment

Date: 2026-10-28

Cleaned summary of the analysis passes that preceded the current notes. No raw dumps.

Timeline of understanding

Period	Focus	Outcome
2026-09-30	Entry, forms, formats, config, render, input, anti-debug	Architecture collapsed; GLEVNW01, drop-files, idle/frame identified
2026-10-02	GS1 dispatch, row table, pattern letters, list flush	DispatchGs1 plated; zlib framing confirmed; version token located
2026-10-03	Session VMT, sign decode, tile walk, debugger runs	SessionRecv cases, DecodeSignText, type-0 vs type-1/2 collision
2026-10-04	List ports, world logins, cross-version tokens	14921/14922/14923 behaviour; GNW03081 construction; world rejections
2026-10-14	Seihai drop-ins, offline slice	Formats locked; script shape known; pics1 blit and CmdCall still open

What remained open at decommission

- Full recovery of type-0 allow lists (GZF lacked PE image)
- Fifteen unplated GS1 opcodes that share the default slot
- Raw gserver pcap (described but never archived)
- List-host S-boxes and remaining needles

These gaps are documented in the Drive zip and in the later cleaned notes of this repository. They are not blockers for the offline slice already exercised by Seihai.

Relationship to Seihai

The cleaned notes here (tile-walk, sign-text, gs1-commands, session-opcodes, config, rendering, offline-training) were written so Seihai could continue without the raw decompile corpus. The Drive zip preserves the original passes for anyone who later needs the full trail.

`client-internals/reference-reimplementations/2dadventureclient.md`

2dadventureclient, behaviour by behaviour

Source: `xtjoeytx/2dadventureclient` at `78040b7` (2020-02-29), read-only. Files: `server/src/TGameWindow.cpp`, `TClient.cpp`, `TAnimation.cpp`, `TImage.cpp`, `TLevel.cpp`, `TPlayer*.cpp`, `bin/data/`. Tags as in `README.md`.

What it is

The tree is GServer-v2 (the 2020 code, with the V8 NPC server) renamed `TServer` to `TClient`, plus an SDL/guichan window. At start it loads the local account `localplayer`, makes a `TPlayer` for it with no socket, marks it local and adds it to the server's own player list **[2dac]**. It still opens the listen socket, so other clients can join it. There is **no client-side network code**: no login builder, no codec, no `PLO_*` handlers. Every "client" behaviour is the server's own object model drawn to a window.

Consequence: it has not been run against real servers as a client, and it cannot tell us anything about the wire from the client's side. What it can show is how GServer-v2 reacts to a client that writes props, which is already covered in `server-notes/`.

Walking and collision

point	reference [2dac]	real clients	verdict
step size	`16 * 0.30` px per axis, truncated to 4 px because the step is an `int`	half a tile (8 px), 1.0 on foot, 1.2 fast (`decomp-findings-2026-10-25-l/01-walking-body-box.md`) **[2.05]**	invented
cadence	one step per drawn frame, frame capped at 60 fps, so about 240 px/s (15 tiles/s) and slower if the frame rate drops	one step per 50 ms, 10 tiles/s (Seihai `walk-pace.md`) **[cap]**	invented
keys	up, left, down, right all summed into one move; facing is the last held key in that order (right wins)	each key is its own tested step, in the order up, left, down, right **[2.05]**	invented
wall test	one point, `(pixelX + move + 32) / 16`, `(pixelY + move + 44) / 16`, so tile (x + 2, y + 2.75)	six points on the leading edge of the box x+0.5..2.5, y+1..3; first four must be free; 0.1-tile corner nudge; snap within 0.0625 **[2.05]**	invented; already recorded in `archival/foot-test-sdl.md`
wall rule	`tiletypes[id] >= 20`, off-board is wall	per-tileset rule; default tileset uses `arrays.dat` lists (`decomp-findings-2026-10-28-b/01-tile-type-tests.md`) **[2.05]**	GServer's table, not the client's rule
water, lava, chairs, beds	`isOnWater` exists, never called by walking	swim, lava burn, sit **[2.05]**	missing
chests, NPCs, other blockers	not tested	chest 2 x 2 and blocking sprites block **[2.05]**	missing
refused step	nothing happens	position put back, "pushing" state starts **[2.05]**	missing

Nothing here is worth adopting. Seihai's body box is the better reference.

Position on the wire

The local player writes props 78/79 (`PLPROP_X2`/`Y2`, pixels, low bit = negative) and then sprite and gani props **[2dac]**. Those are the 2.3+ precise props (`protocol/player-props.md`). 2.05 and 2.22 send half-tile props 15/16 **[2.05][cap]**. The server then derives the half-tile value as `x2 / 16` for older viewers **[srv]**. Keep Seihai's per-version choice.

`gr.x` / `gr.y`

Not a client feature: see `README.md`. The local account here carries `WEAPON -gr_movement` like the stock server's default account **[2dac][srv]**.

Levels, links and gmaps

point	reference [2dac]	real clients	verdict
level formats	the server's loader (GLEVNW01, GR-V1, Z3-V1, gmap)	same set by version (`build-catalogue.md`)	nothing new; see `formats/`
links	never tested for the local player, so doors do nothing	the client tests links on its own walk **[2.05]**	missing
gmap	draws every level of the map at its grid offset	crosses levels by walking (Seihai `gmap_cross` lab runs) **[cap]**	drawing only; walking stops at the level edge because off-board is a wall
camera	centred on the player's level-local pixel position, clamped to the map size	follows the player across the map	the camera ignores the gmap offset, so it is wrong outside the first level

Drawing

point	reference [2dac]	real clients	verdict
tile id to sheet	`col = (id / 512) * 16 + id % 16`, `row = (id % 512) / 16`	the same (`client-assets.md`)	**matches**
tileset	hard-coded `zold_tiles_dusty_01.png`	`pics1` or the level's `addtiledef`	invented for their own world
chests	2 x 2 tiles from sheet cells (56, 15) closed and (29, 19) open; open if the local player's chest list has it	those cells are ids 1784 and 829, the closed and opened chest ids in the 2.22 `arrays.dat` (Seihai `chest-drawing.md`)	**matches**
draw order	players and NPCs together, sorted by pixel y	height layers: `drawunderplayer`, default, players, `drawoverplayer`, `drawaslight` (Seihai `npc-draw-order.md`) **[2.05]**	invented; y-sort would break NPCs that rely on `drawoverplayer`
image files	only `.png` gets a texture; a `.gif` is registered and draws nothing	the classic packages ship `.gif` sheets (1.21 has `letters.gif`, `sprites.gif`, `pics1.gif`); later builds take `.png` too	limitation
names and chat	nickname centred at (x + 24, y + 44), chat at (x + 24, y − 34), TTF font, black 1-px shadow	chat drawn by the name routine, stacked up from the head, no balloon (Seihai `chat-log.md`) **[2.05]**	font and offsets are invented; not checked against the real offsets
HUD	an FPS counter and a test window	`state` sheet HUD (`status-display.md`)	missing

GANI playback

point	reference [2dac]	real clients	verdict
step timer	one timer per loaded `.gani`, shared by every object using it, advanced by `3 * frame time` against `0.05`	each object steps on its own; the wait counter starts at 1 and `WAIT` adds to it (`formats/gani.txt`)	invented
`WAIT`, `PLAYSOUND`	skipped	hold and sound cue	missing
`LOOP`, `CONTINUOUS`, `SETBACKTO`	parsed, not used	used	missing
`ATTR1`, `PARAM`	`ATTR1` draws nothing	attribute sprites (`body-colours-and-attributes.md`)	missing
body colours	none	palette per slot **[2.05]**	missing
`shadow` sprites	drawn at 2/3 alpha	not checked	unknown
placement rows	split on spaces, read as triples with `atoi` (works for `a b c, d e f` only because `atoi("c,")` stops at the comma)	comma-separated placements	accident, don't copy

Seihai's `walk-gani.md` already steps one frame per walk step and lists the real `WAIT` timing as open. The reference does not close that gap.

Chat and commands

Typing in the bottom field sets the chat prop on the local player **[2dac]**. The server's chat commands then run with the local player treated as admin: `warpto` and `.ul` skip the rights checks when the player is local **[2dac]**. That is a convenience of the fork, not a real-client behaviour.

Bottom line

The fork shows GServer-v2 from the inside, which `server-notes/` already covers from the full GServer source. As a client it is an early prototype: every client behaviour it has is simpler than what the 2.05 and 2.22 notes record, and several (y-sorted drawing, shared GANI timers, 15 tiles/s walking) would visibly break the feel if copied. Its two correct details (the tile sheet mapping and the chest cells) Seihai already has.

`client-internals/reference-reimplementations/levelscripteditor.md`

LevelScriptEditor

Source: `xtjoeytx/LevelScriptEditor` at `2a81976` (2021-02-25), read-only, C# WinForms (`src/Levels/*.cs`, `src/Forms/*.cs`, `src/State/GlobalState.cs`). Tags as in `README.md`.

What it is

Not a level editor. It opens a folder, loads every `*.nw` file, lists each level's NPCs in a tree, and lets you edit one NPC's script, image and a description. Its main feature is "replace similar scripts": find every NPC in the folder whose script text is identical (optionally with the same image), and replace them all in one go, with a "mark complete" flag and a "N / M completed" counter **[lse]**. It reads like a tool for a script port (GS1 to GS2 on a big world), not for building levels.

The `.nw` reader

point	reference [lse]	real clients / existing notes	verdict
formats	`.nw` with first line `GLEVNW01` only	2.05 also opens `Z3-V1.03/1.04`, `GR-V1.00–1.03` (`level-editor-205.md`)	narrower
`BOARD x y w layer data`	skips runs outside 0..64 or with `x + w > 64`; keeps one board per layer index	2.22 does not read the layer field: every row goes on the one board (Seihai `level-editor.md`, from the 2.22 notes)	layers are a later-era idea
tile code	`index(c1) << 6 + index(c2)` on `A–Z a–z 0–9 + / =`	same, without = (`formats/glevnw01.txt`)	matches; an invalid character gives a bad id instead of ending the row
`LINK`	level name may contain spaces (everything before the last six tokens)	GServer and the notes say the name has no spaces	lenient, fine
`NPC image x y`	image may contain spaces; x and y as doubles	same; `-` for no image	matches
`SIGN`, `CHEST`	read	read	matches
`BADDY … BADDYEND`	**not read**	written by 2.05 and 2.22	see the writer
anything else (`HEIGHTS` …)	ignored	2.22 has `HEIGHTS` (Seihai `level-editor.md`)	see the writer

The `.nw` writer: lossy

On save the whole file is rebuilt from what was parsed **[lse]**:

- `GLEVNW01`, then 64 `BOARD 0 y 64 layer` rows per layer, then links, signs, chests, NPCs, each with `\n` line ends.
- **Baddies, heights and every unknown line are dropped.** Saving one NPC change deletes every baddy in that level.
- The order is LINK, SIGN, CHEST, NPC. 2.05 writes LINK, SIGN, CHEST, BADDY, NPC (`level-editor-205.md`).
- NPC x and y are written with .NET's default number format, which follows the machine's culture, so on a German or French Windows `30.5` is saved as `30,5`.
- NPC scripts are trimmed of leading and trailing blank space.

Seihai's level tool already went the other way on purpose: its `save` keeps the file's own lines and writes byte-identical output when nothing changed (Seihai `level-editor.md`). Do not take this writer.

NPC header comments

The tool stores its own metadata as comment lines at the top of a script: `//#[MARKED]`, `//#[DESC:text]` **[lse]**. A line starting `// NPC made by` right after them is kept separately and written back. Real clients treat these as plain `//` comments, so they are harmless in a level, but they are this tool's invention, not a Graal convention. If Seihai ever shows them, treat them as comments.

What is worth taking

The workflow, not the code: an index of identical scripts across a level folder, with one edit applied to every copy and a completion count. The `npc-library` already has 3,172 unique scripts; the same grouping would tell a Seihai level author which scripts repeat across GR-V1 levels and which ones are one-offs.

`client-internals/reference-reimplementations/README.md`

Reference reimplementations: what to trust

Date: 2026-10-30. Read-only study of two xtjoeytx repositories, weighed against the real-client notes in this repository and Seihai's own documentation. The code is summarized and cited, not copied. Nothing was sent to xtjoeytx (no issue, PR or fork).

repo	commit read	what it really is
`xtjoeytx/2dadventureclient`	`78040b7` (2020-02-29)	GServer-v2 (2020) with an SDL window and one local player bolted on. It hosts a server; it never connects to one. See `2dadventureclient.md`.
`xtjoeytx/LevelScriptEditor`	`2a81976` (2021-02-25)	A C# tool for bulk-editing NPC scripts in a folder of `.nw` levels, with "mark complete" tracking. Not a tile editor. See `levelscripteditor.md`.

Tags used in these notes: **[2dac]** 2dadventureclient source, **[lse]** LevelScriptEditor source, **[srv]** GServer-v2 source, **[2.05]**/**[2.22]** real-client notes already in this repository, **[cap]** lab captures, **[inf]** inferred.

Verdict in one table

behaviour	reference does	real clients	verdict for Seihai
wall test	one point at (x+2, y+2.75)	six-probe box, corner nudge, flush snap	**invented**, keep Seihai's box
walk speed	4 px per rendered frame (about 15 tiles/s at 60 fps)	0.5 tile per 50 ms (10 tiles/s)	**invented**, keep 50 ms steps
diagonal walking	both axes, one test, last key wins the facing	up, left, down, right as separate tested steps	**invented**
position on the wire	pixel props 78/79 for every client	2.05/2.22 send half-tile props 15/16; pixels from 2.3	**2.3+ only**, keep per-version
`gr.x` / `gr.y` flags	sent by a **server-supplied weapon** `-gr_movement`	—	**correction** to Seihai's `gr-flags.md`, see below
tile id to sheet pixel	512-id blocks of 16 x 32	same	**matches**
chest picture	tiles 1784 (closed) and 829 (open)	same ids in 2.22 `arrays.dat`	**matches**
draw order	all players and NPCs sorted by y	height layers (under, player, over, light)	**invented**
GANI timing	one shared timer per file, `WAIT` and `PLAYSOUND` skipped	per-object counter, `WAIT` holds	**incomplete**, don't copy
level links, gmap crossing, water, lava	not done for the local player	done	**missing**, nothing to learn
`.nw` read/write (LSE)	GLEVNW01 only, BADDY dropped on save	2.05/2.22 write BADDY	**lossy**, don't copy the writer
NPC metadata headers (LSE)	`//#[KEY:value]` comment lines	not a client feature	**invented**, harmless comments
multi-layer boards (LSE)	keeps each `BOARD` layer apart	2.22 ignores the layer field	**later era**, keep Seihai's merge

The correction: `gr.x` and `gr.y` come from the server

Seihai's `documentation/gr-flags.md` says "real clients keep the player's position in two client flags, `gr.x` and `gr.y`" and models it as a client behaviour per version (`LoginOnly` for 2.17, `Tracking` for 2.22). The source of those flags is the server:

- GServer-v2 ships a weapon `-gr_movement` (`bin/servers/default/weapons/weapon-gr_movement.txt`) and puts `WEAPON -gr_movement` in `defaultaccount.txt`, so every new account on a stock server gets it **[srv]**. The 2dadventureclient fork carries the same files **[2dac]**.
- The weapon is a clientside GS1 script. On `playerenters` or `timeout` it does `setstring gr.x,#v(int((playerx+0.25)*2)/2)` (and the same for `gr.y`), then `timeout=0.05`. Unless the flag `gr.off`, `gr.x.off` or `gr.y.off` is set **[srv]**.
- The server's flag handler swallows `gr.x`/`gr.y`/`gr.z` from clients below 2.3 and turns them into props 15/16/(z) for the other players, but only on ticks where no real movement prop arrived. The readme says this is to "simulate the smooth movement as found in the Graal clients 2.3 and up" **[srv]**. The setting is `flaghack_movement` (default on).
- The lab runs stock GServer-v2 (`tools/lab-setup.md`), and the public GServer-RS run also sent `-gr_movement` in the login burst (`client-internals/decomp-findings-2026-10-25-l/04-public-server-run.md`).

So what the captures show fits a script running in the client:

- the value is the position rounded to half a tile, written without decimals for whole numbers (`#v` formatting) **[cap]**;
- a 50 ms cadence in 2.22 is the `timeout=0.05` loop;
- 2.17 sending once at login fits `playerenters` firing and the timeout loop not running there **[inf]**;
- 2.31 never sending is not explained by the script; the cause was not checked **[inf]**.

For Seihai: a client should not send `gr.x`/`gr.y` on its own. It should send them when, and only when, it runs a weapon script that sets them. On an official 2001-era server there is no such weapon, so a real 2.05 or 2.22 client would send nothing. Seihai's script engine already runs weapon scripts (the `§` line-break fix in Seihai's `public-gserver-run.md`), so the hard-coded `protocol.gr_flags` profile switch is at best a duplicate of that, and at worst a double send on a GServer-v2 server. Worth a lab run with the weapon removed from the lab account to confirm the real 2.22 then sends no `gr` flags.

What Seihai could adopt

Little of the client code, because the real-client notes here are already more precise. The useful parts:

1. **The `-gr_movement` explanation** above (a fix, not an adoption).
2. **GServer's version workarounds as a checklist of server quirks** a Seihai client will meet on GServer-v2: the server deletes `Bomb` and `Bow` right after login because "the client adds them", sends `-gr_zlib_fix` to 2.21 to 2.31, and pushes every big-map file to 2.31 because that client does not request it after `setmap` **[srv]**. These are server behaviours, not client ones, so a nostalgic Seihai should tolerate them, not imitate them.
3. **LevelScriptEditor's find-identical-scripts idea** for Seihai tooling: group NPCs across a level folder by exact script text (optionally the same image), edit once, apply to all. This fits the `npc-library` work. Take the idea, not the writer (see `levelscripteditor.md`).

Small thing found in passing (Seihai)

`client/src/assets/level.rs` still has `tile_src_rect`, which maps a tile id row-major on a 128-wide sheet (`col = id % 128`). That is the mapping `client-assets.md` corrected on 2026-10-25. Nothing calls it (the renderer in `render/tiles.rs` uses the right block mapping), so it is dead code that could mislead.

`client-internals/relay-room-bridge.md`

Java Graal meets the Delphi betas: a relay room bridge (2026-10-31)

Question: can Java Graal (July 2000) and 5th Graal (October 1998) play in one room? Both talk to Stefan Knorr's GServ relay lobby (`protocol/relay-1998.md`), but they join different rooms. This note runs them, and 4th Graal alpha, live against Knorr's own GServ, and tries a proxy that renames rooms.

Tags: **[live]** seen in running clients or on the wire, **[bin]** read from the binaries' strings. No decompiler output is reproduced.

Result

Pair	Relay	Meet?	What went across
Java Graal + 4th Graal alpha	stock GServ	**yes** [live]	both drawn, nicknames, chat both ways
Java Graal + 5th Graal	stock GServ	no [bin, live rooms]	different rooms (`G2<level>` vs `G3<level>`)
Java Graal + 5th Graal	GServ behind the bridge, `G2` → `G3`	**yes** [live]	both drawn, nicknames, walking both ways (5th Graal's from run 4), chat both ways, a level change and back

So the "Java Graal speaks 4th alpha's dialect" reading in `precursors-and-fan-builds.md` holds on the wire, and its last open point is closed: 4th alpha joins `G2` + the bare level file name (`G2level13.graal`), the same string the applet builds. For every later beta a one-line rename in the join is enough to put them together; the `D` payloads already line up.

Rooms each build joins

Every Delphi beta opens **two** connections to the relay: one to a room per level, one to a fixed game-wide room. The Java builds open one.

Build	Level room	Game-wide room	Basis
1st Graal	`Zhouse12` (`Z` + level, no extension)	`Zelda2`	[live]
2nd Graal	as 1st	`Zelda2`	[bin]
3rd Graal	`Ghouse1.graal` (`G` + level file)	`Graal`	[live]
4th Graal	`G<level>`	`Graal`	[bin]
4th Graal alpha	`G2level13.graal`	`Graal2`	[live]
5th Graal	`G3level13.graal`	`Graal3`	[live]
5th alpha, 6th to 1.1.9	`G3` … `G8` + level	`Graal3` … `Graal8`	[bin] (1.1.9 also in `live-trace-early.md`)
Java Graal	`G2level13.graal`	none (a `Graal2` mode exists but is never selected)	[live]
Zelda Online 2	`Zlevel11.6` (`Z` + base + `1.6`)	none	[live], 2026-10-30

1st Graal's `Zhouse12` is either `Z` + `house12` or `Z` + `house1` + `2` (the `Z` + base + `2` branch in `early-clients.md`); one run does not tell them apart. Either way it never equals Zelda Online 2's `Z<base>1.6`.

How it was run

- **Server:** Zelda Online 2's `server/GServer.java` (Knorr's GServ Standalone), compiled with a current JDK and run unchanged on 14801.
- **Bridge:** `tools/relay-bridge/room_bridge.py` on 14802, in front of GServ. It passes every line unchanged except `N<room>#<name>` / `O<room>#<name>`, where a rule `OLD=NEW` swaps the room prefix (`G2=G3` turns `G2level13.graal` into `G3level13.graal`). It logs every line both ways. With no rule it is a plain logging proxy.
- **Java Graal:** the 2002 rip's `Graal.jar` (the loose class files in the rip are truncated), served over HTTP as the applet's codebase and started with `tools/relay-bridge/AppletRun.java`, a small `AppletStub` frame. The applet dials its codebase host on 14802, so it always goes through the bridge.
- **Delphi betas:** the shipped `Graal.exe` under Wine 9 (Xvfb, openbox, the Wine virtual desktop), started with `WINEDLLOVERRIDES="dinput=d"` so the arrow keys work (see run 4). `tools/relay-bridge/run_delphi_beta.sh` sets this up. From 3rd Graal on the server box is greyed out at Cyberjoueur's dialog address on `<loopback-outside>:14802` (the historical fixed IPv4, remapped onto the loopback interface as a fake outside address so the beta reached the bridge). 4th alpha and 1st Graal keep an editable the loopback address on 14801 and went straight to GServ (captured with tcpdump, decoded with `tools/relay-bridge/pcap_lines.py`).

Captures and screenshots: `live-runs/relay-bridge-2026-10-31/`.

Run 1: Java Graal and 5th Graal through the bridge

```
room_bridge.py --listen 14802 --server <loopback>:14801 --map G2=G3
```

- Java Graal joined `G2level13.graal`; the bridge sent `NG3level13.graal#def`. 5th Graal joined `G3level13.graal` and `Graal3`. GServ announced each to the other (`+`), both sent `P`, and the state burst followed.
- 5th Graal drew the Java player, its walking (a run of `P` lines) and its chat (`hi from java`). Java Graal drew the 5th Graal player with its nickname (`Fifth`) and chat (`hello from 5th`). (`run1-java-to-5th.png`, `run1-5th-to-java.png`.)
- Java Graal walked into `house10.graal` and back: it sent `C` and `-`, then joined `G2house10.graal` (bridged to `G3house10.graal`), and on return `G3level13.graal` again, and the 5th Graal player was there again.
- `D` types sent in the run: Java Graal `T P C N D`; 5th Graal `T P Y C N S`. Java Graal ignores `Y` and `S` (colours), so the 5th Graal player shows in default colours there. Nothing broke on either side.
- 5th Graal's own walking was not seen in this run: its arrow keys did not reach the game under Wine, so its player stood still. Run 4 fixes that and shows the walking.

Run 4: 5th Graal walks in the shared room

Same setup as run 1 (`--map G2=G3`), with 5th Graal started under `WINEDLLOVERRIDES="dinput=d"`.

- **Why the keys were lost.** 5th Graal reads the keyboard through DelphiX `TDXInput`, which loads DirectInput. Wine's `+dinput` trace shows the keyboard data format it sets: `dwDataSize 256, dwNumObjs 1`, one object at offset 1 (`run4-dinput-format.txt`). Windows fills the whole 256-byte key buffer anyway. Wine fills only the objects listed, so the arrow-key bytes stay zero.

Chat typing and mouse clicks arrive as window messages, which is why they worked. This is the same fault as in the 1.x clients (`live-trace-early.md`, `live-gserver13.md`).

- **The fix.** With `dinput=d` the DirectInput load fails, DelphiX falls back to `GetKeyboardState`, and `xdotool keydown` reaches the game. This is the setting the 2026-10-30 six-era runs used (`version-profiles/matrix/live-runs-2026-10-30.md`). No change to the exe and no memory patch is needed. The game window must have focus (`xdotool windowactivate`/`windowfocus`), which the openbox + virtual desktop setup allows.
- **Before:** without the override, holding Right for 1.5 s sent nothing. **After:** holding Right, Down and Left sent a stream of `D #P…` lines, one every 100 ms while a key was held, each relayed to Java Graal (`run4-5th-java-walk-bridge.log`). [live]
- Java Graal drew the `Fifth` player walking right across the level13 yard (`run4-5th-walks-java-view.png`, before and after). [live]
- 5th Graal's own screen often stops repainting the board under Wine (as noted for 2.05 in `board-respawn.md`), so judge its movement from the wire or from the other client, not from its own window.

Run 2: Java Graal and 4th Graal alpha, no rename

Bridge with no rules (Java side), 4th alpha straight to GServ.

- 4th alpha joined `G2level13.graal` and `Graal2`; Java Graal joined `G2level13.graal`. Same room, so GServ paired them with no help.
- Chat went both ways (`java meets fourth`, `fourth says hi`), and each drew the other with its nickname (`run2-4thalpha-java-chat.png`).
- 4th alpha sent `T P Y C N S`, Java Graal `T P C N D`, as in run 1.

Run 3: rooms of 3rd and 1st Graal

Only the joins were needed: 3rd Graal sent `NGhouse1.graal#def` and `NGraal#def`; 1st Graal sent `NZhouse12#def` and `NZelda2#def`.

What this means

- A relay that serves these builds together needs only the join rename. The `D` lines are already the same dialect from Zelda Online 2 through 1.1.9, and each side ignores the types it does not know.
- The rename must keep the level part: mapping a level room onto a game-wide room (for example `G2<level>` onto `Graal3`) would put players on different levels in one room.
- Bridging other pairs is the same rule with other prefixes (`G2=G5` for Java Graal with 7th to 9th Graal, and so on). Only 4th alpha and 5th Graal were run.
- Seihai lists all of these builds as offline-only (it has no relay client); its rows carry the rooms above (`documentation/offline-only-builds.md` in Seihai).

`client-internals/rendering-observations.md`

Rendering observations

Date: 2026-10-14

Derived from observed client behavior and public DirectX notes. No decompile material.

Frame loop

The client runs an idle handler that gates on a 50 ms timer, then a frame function that handles clock skew, network traffic, and draw. A stall is one frame; there is no catch-up burst. The frame function ignores the step count that the pacer may supply.

Draw path

The frame function calls a draw routine that uses the currently selected tile sheet. The device object exposes:

- viewport
- material
- projection transform
- ambient render state (set to white)

Supported image suffixes observed in the drop path are `.txt`, `.gif`, `.png`, and `.mng`. The default sheet is `pics1.png`.

Tile sheet and blit

The active sheet is selected from the loaded definition list when the level is entered. An offline session that already draws an ASCII or PPM view of the board can replace that view with a blit of the selected sheet once:

1. The sheet is chosen (default `pics1.png`, type 0).
2. The type-0 allow lists are applied so only walkable tiles are treated as open.
3. The stamp vocabulary is used so a named stamp expands to the correct tile ids.

A single stamp blit (for example the 2×2 Bush) is enough to prove the path. After that, the full board can be blitted from the same sheet.

Offline implication

The next useful increment remains small:

1. Blit one stamp from `pics1` using the sheet the original client would have selected.
2. Wire `playertouchsme` so that a touch actually hides the NPC or warps the player.

After those two changes the offline session starts to look like a level instead of a parser. Both steps use data and rules already documented in `formats/tile-walk.md` and `npc-library/gs1-commands.md`.